



EZtCloud 文档中心

客户交付版

文档类型: 产品使用与接口文档

生成日期: 2026-03-27

目录

文档约定

EZtCloud是什么？

EZtCloud核心概念

设备联网方式

设备接入协议

设备快速连接

设备MQTT接入协议

一级子设备MQTT接入协议

子设备MQTT接入协议

设备TCP接入协议

设备通过DTU接入

设备消息调试

创建设备类型

功能定义

自定义数据流

Modbus寄存器设置

个性化UI

创建规则引擎

属性上报预处理

属性上报

事件上报

自定义数据上报

云函数内置函数库

创建任务

属性下发任务

命令下发任务

自定义下发任务

告警规则

告警通知

告警历史

OTA固件升级

API 概述

HTTPS API

MQTT API

Topic映射

文档约定

文档约定

文本块说明

这是一个提示示例：

 **提示**

这是提示内容，为用户提供有用的建议。

这是一个注意示例：

 **注意**

这是注意内容，用于提醒用户需要注意的重要事项。

这是一个危险示例：

 **危险**

这是危险内容，警告用户潜在的风险。

EZtCloud是什么？

EZtCloud是什么？

EZtCloud是物联网设备统一接入平台，EZtCloud可接入各类网关，传感器、执行器、控制器、智能硬件等，实现数据采集、远程控制，数据分析、告警通知、智能联动。同时还开放API和实时消息，能够和其它系统集成对接。

通过使用EZtCloud，企业可以大大缩短搭建物联网系统的时间，节省软件开发费用，降低定制开发的风险，快速落地数字化和智能化项目。



EZtCloud既可以配合传感器、网关、DTU等设备快速搭建一套数据采集方案，也可以帮助智能硬件接入云端实现远程控制或设备联动等功能，还可以与多种硬件结合实现物联网行业解决方案，并为物联网应用提供了无限可能。

EZtCloud的用户遍布多个行业和应用场景，包括但不限于：

- 工业物联网
- 智慧能源
- 智能制造
- 智能家居
- 智慧农业
- 智能养殖
- 智慧楼宇
- 智慧园区
- 智慧物流
- 智慧城市
- 资产管理

EZtCloud核心概念

EZtCloud核心概念

项目

项目 是 EZtCloud公有云中资产的组织单位，在开始任何操作之前，您首先要创建一个项目。

每个项目下的资产是完全隔离的，互不影响的，而且项目本身之间不能直接互相访问。当然，您完全可以通过项目 API，分别对多个项目进行数据访问，在应用端进行数据整合或联动控制。

设备类型

在一个项目下，可以创建多个设备类型。即，设备类型仅在其所属的某个特定项目内有意义。

一个**设备类型**对应于现实世界中某项目下的某一型设备。

设备类型不仅可以对设备进行 **功能定义** 和 **消息合法性验证**，还可以将一系列增强的能力附加给绑定到设备类型的所有设备。

这些附加能力包括：

- 支持自定义数据通信，包括二进制、JSON、Plaintext等格式的自定义消息。
- 让设备支持TCP接入，用在一些仅提供TCP连网功能的设备或DTU。
- 在浏览设备数据时，显示更具有可读性的属性名称，以及单位等。
- 为大量同类设备提供规则、任务、告警通知等全局设置。
- 将设备类型的功能定义、规则、任务等配置发布到 **公共产品库**，可帮助客户或合作伙伴快速复用设备类型相关配置，也有利于在其它项目中快速重用。
- 为设备创建 OTA 固件远程升级。

设备

用户可以基于设备类型，创建1个或多个设备。

EZtCloud中的一个**设备**对应现实世界中的一台物联网设备。

按设备自身是否具备接入云平台的能力来划分，即，按 **接入类型** 划分，所有设备分为两类：直接上网设备 和 间接上网设备。

直接上网设备

设备自身具备直接接入云平台的能力。即，这类设备可以通过以太网、蜂窝网络等**直接**与云平台通讯。

间接上网设备

设备自身**不具备**直接接入云平台的能力。这类设备需要只能通过**父设备**接入EZtCloud。

子设备

由于 间接上网设备 自身不具备直接接入云平台的能力，此类设备只能借助其它设备实现与云平台的间接通讯。此时，我们称这个间接与云平台通讯的设备为其所借助设备的**子设备**。

在EZtCloud中，任何设备（间接上网设备 或 直接上网设备）下，都可配置0个或N个子设备。

由于 直接上网设备 本身具备与云平台直接通讯的能力，故其不可作为任何设备的子设备。

父设备

若设备A的子设备中包含B，则我们称A为B的**父设备**。

厂商

厂商 是指某现实世界中设备的生产厂商。

厂商是独立于项目存在的。

厂商的管理员通常更熟悉其生产的某型号产品，他们可以将相关配置保存为一个 公共产品，以便于所有用户使用。

公共产品

公共产品 可被看作一个设备类型的模板。任何用户可在其项目下，基于公共产品快速创建一个设备类型。

设备类型中的所有配置都可保存在公共产品中。

公共产品库

公共产品库 是 公共产品 的集合，它对所有用户可见。厂商 的成员用户可自行发布其 公共产品 到 **公共产品库**。

设备联网方式

设备联网方式

设备和云平台要实现通信，首先要确保设备可以连网，也就是让设备接入互联网，包括设备直接连网和通过父设备连网。介绍设备常用的几种联网方式：

WiFi

通过WiFi芯片或模组，设备可以接入WiFi热点（AP），后者通常也是可以接入互联网或者私有网络的路由器，使得设备可以直接使用TCP/IP协议和云平台建立通信。

💡 提示

WiFi和以下几种连网方式，都属于链路层通信协议，它为设备和云平台建立通信管道，而在管道之上的数据通信协议，也就是后边提到的接入协议，可以有多种选择，比如 MQTT、HTTP、TCP 等，在其他章节会有介绍。

以太网

通过以太网接入互联网的方式很常见，就像传统的PC电脑通过网线接入路由器实现上网一样。以太网支持10M/100M/1000M 的传输速率，通过IP路由实现网络互通，最终速率受限于整条链路中最慢的交换节点。

蜂窝网络

蜂窝网络由运营商提供，设备通过集成通信模组来使用相应的网络。目前在中国主要使用2G/3G/4G/5G/NB-IOT几种蜂窝网络。

通过父设备

以上的几种连网方式，都是在设备上集成通信模组，使设备实现直接接入EZtCloud，这类设备我们称之为 [直接上网设备](#)。除此之外，有些设备自身不具备上网能力，只能通过**父设备**接入EZtCloud，这类设备我们称之为 [间接上网设备](#)。

在物联网领域中，网关通常扮演者父设备的角色，它的作用是将**间接上网设备**，通过父设备的中转，接入云平台。父设备起到的作用是数据转发和协议转换。

除了通信中转作用，父设备还具有另一个很重要的作用，那就是业务层的边缘计算，包括对设备的运维管理、联动控制、数据缓存等。

凡是实现以上作用的硬件或软件都可以称为**父设备**。

那些和父设备连接的设备，我们也称为子设备。在EZtCloud中，任何设备都可被作为父设备，即任何设备下，都可配置0个或N个子设备。而**直接上网设备**不可作为子设备。

那么，子设备和父设备都有哪些连接方式和通信协议呢？连接方式非常多，有线方式包括UART、USB、I2C、SPI、RS485、RS232、CAN、PROFIBUS、以太网、各类工业协议等，无线方式包括 WiFi、蓝牙、Zigbee、Lora、其它无线通信等。

以上这些都是链路层通信协议，在此之上，我们要知道消息包的数据格式，以及它们代表什么含义，我们还需要应用层通信协议，比如 Modbus、BACnet，以及一些工业级国标协议等。

通过DTU

DTU是Data Transfer Unit的简称，也就是数据传输单元。DTU就是一种典型的网关产品，它的特点在于面向用户的快速配置能力，部分DTU还支持可编程。DTU 将应用到更多的物联网场景中，成为一种开箱即用的硬件网关。

DTU同样也是使用前边提到的几种常用的连网方式，而DTU与云平台的接入方式，常见的是 ***MQTT 和 TCP***，这里的TCP方式更多是采用透传方式，也就是DTU 将子设备发来的消息不做任何业务层的处理，直接封装成TCP/IP消息包，转发给平台，同时也将平台下发的数据包，经过链路协议层的处理，转发给子设备。

设备接入协议

设备接入协议

设备和云平台如何交换数据，这就涉及到设备和云平台的接入通信协议。

这一节，将简要介绍云平台支持的设备接入协议，帮您对这些方式有一个总体的了解，并大概知道如何选择它们。每一种接入方式的具体原理和操作方式，我们将在后续的相关章节中详细说明。

MQTT协议

MQTT 是物联网领域常见的通讯协议。在通过 MQTT 接入时，设备需要和云平台建立 TCP 长连接，并通过 MQTT 协议特有的方式完成身份认证。当设备成功连接到云平台后，通过发布（Publish）和订阅（Subscribe）相应的主题（Topic），来完成与云平台的消息通信。

若采用 MQTT 方式接入，设备首先要和云平台建立 MQTT 连接，您可在[设备详情页的连接](#)tab页找到 MQTT 连接的主机地址、端口号、Username 和 Password，如图：

MQTT 接入点

mqtt://mqtt.eztcloud.com:7883

MQTT 主机： mqtt.eztcloud.com

MQTT 端口： 7883

Username： DU_J8PrURek04

Password：

当设备与云平台成功连接 MQTT 后，设备就可以按照 [设备MQTT接入协议](#) ****收发数据。

对于子设备通过父设备接入云平台的场景，父设备可按照 [子设备MQTT接入协议](#) 向平台收发其子设备数据。

注意

[一级子设备MQTT接入协议](#) 作为 [子设备MQTT接入协议](#) 的一个子集，已不推荐使用，所有新接入设备应优先考虑使用后者。

EZtCloud作为物联网PaaS云平台，对设备 MQTT 接入提供了内置的访问协议规范，让设备和云平台的消息通信更加有章可循，大大简化了物联网项目的开发难度，缩短了产品的开发周期。

不同于普通的 MQTT 使用方式，我们提供了标准的内置主题，这足以实现绝大多数的物联网应用场景。

而对于另一些个性化的消息通信场景，我们也提供了灵活的自定义主题，您可以创建自己的主题，并配合包括云函数在内的规则引擎，实现任何消息通信需求。

注意

全局的，除 [自定义数据流](#) 外，所有 MQTT 负载消息都必须是 JSON 格式，如果发布的不是 JSON 格式，设备会被平台主动断开 MQTT 连接。

TCP协议

除此之外，平台还支持TCP接入协议，在控制台可以设置基于TCP的自定义数据格式，支持 ASCII 、 HEX、JSON 等格式。

常用的通讯设备DTU，通常都支持 TCP 透传方式接入云平台。

设备快速连接

设备快速连接

在EZtCloud中，设备快速连接的核心思路是：设备上电时，可通过“调用一个HTTP API”的方式，获取到 [连接信息](#)，设备根据该信息，连接到 EZtCloud。

本文档描述了设备快速连接到EZtCloud的两种场景及其实现方式。

连接场景

EZtCloud支持如下两种场景下的设备快速连接：

- [场景一：设备上电后直接接入指定项目](#)
- [场景二：用户扫码添加设备](#)

当设备具备写入变量 ProjectKey 的能力时，推荐优先采用前者，其优点是设备上电即自动连接EZtCloud，全程无需人工干预。当设备不具备上述能力时，则可采用后者。

场景一：设备上电后直接接入指定项目

前提条件

- 设备需具备写入变量 ProjectKey 的能力。这种能力的表现形式不限，如：生产烧录设备时写入、通过设备自带的液晶屏和按键写入、通过上位机软件+串口线写入、通过专用设备写入等。

提示

可在 [控制台](#) > [项目设置](#) > [应用层API](#) 中获取 ProjectKey。

连接流程

1. 设备上电后，通过 [场景一的获取连接信息接口](#) 获取 [连接信息](#)。
2. 设备使用获取到的 [连接信息](#) 连接到云端。

获取连接信息API

当项目中不存在该设备时，则自动创建该设备。

进一步的，当项目中不存在该设备类型时，则自动创建该设备类型。

说明

为了兼容尽可能多的设备，下述接口同时支持请求参数传参 和 请求体传参，其效果相同。两者的并集将作为最终的传参，两者同时传入时，以请求体参数为准。

为了兼容尽可能多的设备，下述接口同时支持 POST 和 GET方法，其效果相同。受限于HTTP协议，当采用GET方法时，不支持请求体传参。

请求信息

- URL: <HTTPS API 接入点>/api/connect/1/?project=<project>&product=<product>&sn=<sn> (HTTPS API 接入点可从如下页面获取: 控制台 > 项目设置 > 应用层API。URL样例: https://api.eztcloud.com/api/connect/1/?project=PR_q9p7maKXgY&product=PK_kkfGM&sn=1000000066)
- 方法: POST 或 GET
- 需要认证或鉴权: 否
- 熔断频率: 基于IP, 1000次/小时

请求参数

- project: ProjectKey, 可从如下页面获取: 控制台 > 项目设置 > 应用层API
- product: 产品识别码 或 产品Key
- sn: 设备唯一标识。用于标识具体某一台硬件设备, 通常被硬件厂家标识为SN、EUI等

请求体

```
{
  "project": "ProjectKey",
  "product": "产品识别码 或 产品Key",
  "sn": "设备SN"
}
```

响应

- 状态码 420: 项目不存在
- 状态码 436: 产品识别码不存在
- 状态码 200: 成功, 返回 [连接信息](#)

场景二: 用户扫码添加设备

前提条件

每个设备需具备唯一的二维码, 格式为如下二者之一:

格式1:

```
http://qr.eztcloud.com/?product=<product>&sn=<sn>
```

格式2:

```
<product>&<sn>
```

⚠ 注意

尽可能使用 格式1, 它具有扫码拉起EztCloud移动端应用的能力。格式2 仅在设备太小, 用于展示二维码的空间紧张时使用。

连接流程

1. 用户通过移动端登录项目, 扫描设备二维码。
2. 设备上电后, 通过 [场景二的获取连接信息接口](#) 获取 [连接信息](#)。
3. 设备使用获取到的 [连接信息](#) 连接到云端。

获取连接信息API

当项目中不存在该设备时，则返回错误码 401。

说明

为了兼容尽可能多的设备，下述接口同时支持请求参数传参 和 请求体传参，其效果相同。两者的并集将作为最终的传参，两者同时传入时，以请求体参数为准。

为了兼容尽可能多的设备，下述接口同时支持 POST 和 GET方法，其效果相同。受限于HTTP协议，当采用GET方法时，不支持请求体传参。

请求信息

- URL: <HTTPS API 接入点>/api/connect/2/?product=<product>&sn=<sn> (HTTPS API 接入点可从如下页面获取：
控制台 > 项目设置 > 应用层API 。URL样例：https://api.eztcloud.com/api/connect/2/?product=PK_kkfGM&sn=1000000066)
- 方法: POST 或 GET
- 需要认证或鉴权: 否
- 熔断频率: 基于IP，1000次/每小时

请求参数

- product: 产品识别码 或 产品Key
- sn: 设备唯一标识。用于标识具体某一台硬件设备，通常被硬件厂家标识为SN、EUI等

请求体

```
{
  "product": "产品识别码 或 产品Key",
  "sn": "设备SN"
}
```

响应

- 状态码 436: 产品识别码不存在
- 状态码 401: 设备不存在
- 状态码 200: 成功，返回 [连接信息](#)

连接信息

成功响应将返回如下格式的连接信息：

```
{
  "mqtt": {
    "host": "mqtt.eztcloud.com",
    "port": 7883,
    "username": "DU_JEwVTAmL3m",
    "password": "1234567890"
  },
  "mqttps": {
    "host": "mqtt.eztcloud.com",
```

```
"port": 7885,
"username": "DU_JEwVTAmL3m",
"password": "1234567890"
},
"tcp": {
  "host": "tcp.eztcloud.com",
  "port": 6879,
  "reg": "DU_JEwVTAmL3m&1234567890"
},
"topic": {
  // 设备上报属性
  "attributes": "attributes/DU_JEwVTAmL3m",

  // 设备上报属性的响应
  "attributes_response": "attributes_response/DU_JEwVTAmL3m",

  // 设备获取云端属性
  "attributes_get": "attributes_get/DU_JEwVTAmL3m",

  // 设备获取云端属性的响应
  "attributes_get_response": "attributes_get_response/DU_JEwVTAmL3m",

  // 云端下发属性至设备
  "attributes_push": "attributes_push/DU_JEwVTAmL3m",

  // 设备上报事件
  "event_report": "event_report/DU_JEwVTAmL3m",

  // 设备上报事件的响应
  "event_response": "event_response/DU_JEwVTAmL3m",

  // 云端下发命令至设备
  "command_send": "command_send/DU_JEwVTAmL3m",

  // 云端下发命令至设备的响应
  "command_reply": "command_reply/DU_JEwVTAmL3m",

  // 子设备上报上线通知
  "sub_connect": "gateway_connect/DU_JEwVTAmL3m",

  // 子设备上报下线通知
  "sub_disconnect": "gateway_disconnect/DU_JEwVTAmL3m",

  // 子设备上报属性
  "sub_attributes": "gateway_attributes/DU_JEwVTAmL3m",

  // 子设备上报属性的响应
  "sub_attributes_response": "gateway_attributes_response/DU_JEwVTAmL3m",

  // 子设备获取云端属性
  "sub_attributes_get": "gateway_attributes_get/DU_JEwVTAmL3m",
```

```
// 子设备获取云端属性的响应
"sub_attributes_get_response": "gateway_attributes_get_response/DU_JEwVTAmL3m",

// 云端下发属性至子设备
"sub_attributes_push": "gateway_attributes_push/DU_JEwVTAmL3m",

// 子设备上报事件
"sub_event_report": "gateway_event_report/DU_JEwVTAmL3m",

// 子设备上报事件的响应
"sub_event_response": "gateway_event_response/DU_JEwVTAmL3m",

// 云端下发命令至子设备
"sub_command_send": "gateway_command_send/DU_JEwVTAmL3m",

// 云端下发命令至子设备的响应
"sub_command_reply": "gateway_command_reply/DU_JEwVTAmL3m"
}
}
```

设备可以根据自身支持的协议类型，选择相应的连接信息进行连接。

上述为默认响应格式，必要时，可通过如下方式自定义响应格式：

1. 基于公共产品库创建设备类型。
2. 打开上述设备类型的详情页，点击 **设置** 选项卡，点击 **设备连接信息** 并编辑保存。

设备MQTT接入协议

设备MQTT接入协议

主题概览

以下主题用于设备与云端通讯：

消息类型	主题	发布/订阅
设备上报属性	attributes/{username}	设备发布
设备上报属性的响应	attributes_response/{username}	设备订阅
设备获取云端属性	attributes_get/{username}	设备发布
设备获取云端属性的响应	attributes_get_response/{username}	设备订阅
云端下发属性至设备	attributes_push/{username}	设备订阅
设备上报事件	event_report/{username}	设备发布
设备上报事件的响应	event_response/{username}	设备订阅
云端下发命令至设备	command_send/{username}	设备订阅
云端下发命令至设备的响应	command_reply/{username}	设备发布
设备上报自定义数据	data/{username}/{identifier}	设备发布
云端下发自定义数据至设备	data_set/{username}/{identifier}	设备订阅

💡 提示

以上自定义数据相关主题中的 identifier，是指自定义数据流标识符。

各主题的使用方法

下面我们逐个介绍各个主题的使用方法。

设备上报属性

设备上报属性使用如下主题：

attributes/{username}

一个简单的属性上报消息如下：

```
{
  "temperature": 28.5,
  "light": 2000,
```

```
"switch": true
}
```

设备发布以上消息后，控制台的设备详情页会实时更新显示设备属性的最新值。

设备上报属性的响应

如果我们想知道属性上报是否被云平台成功接受，设备可以订阅如下主题：

attributes_response/{username}

当设备上报属性后，便会通过该主题收到来自云平台的响应消息，如果云平台接收成功，响应消息如下：

```
{
  "error_code": 0,
  "error_msg": "OK",
  "ts": 1750234507181
}
```

如果云平台未成功接收，响应消息中会包含错误原因，例如：

```
{
  "error_code": -1,
  "error_msg": "Device message frequency too fast, please wait for a moment",
  "ts": 1750234507181
}
```

该错误表示属性上报间隔太短，所以云平台会自动忽略过于频繁的属性上报消息。

注意

此响应消息默认不开启，需要时可在此处开启：设备类型详情页 > 设置 > 云端响应。

设备获取云端属性

当设备希望从云平台获取属性当前值时，发送消息到如下主题：

attributes_get/{username}

消息内容格式如下：

```
{
  "keys": []
}
```

这表示获取所有属性，也可以指定个别属性，如下：

```
{
  "keys": ["temperature", "humidity"]
}
```

设备获取云端属性的响应

请确保设备已经订阅了如下主题：

attributes_get_response/{username}

当设备发送获取属性的消息后，便会通过以上订阅主题，收到如下的响应消息：

```
{
  "error_code": 0,
  "error_msg": "",
  "attributes": {
    "sz": 3
  },
  "ts": 1750234507181
}
```

云端下发属性至设备

除了设备主动获取属性以外，我们也需要让设备可以实时接收云平台下发的属性。

确保设备已订阅如下主题：

attributes_push/{username}

当云平台下发属性给设备时，设备会通过以上订阅主题，收到 JSON 结构的消息，例如：

```
{
  "switch": false
}
```

这个示例消息显然是通知设备关闭某个开关，设备通过自身的程序实现该功能后，可以接着向云平台上报属性或上报事件，让云平台得到确认。

设备上报事件

通过事件，设备可以向云端报告消息，而不需要上报任何属性。

一个事件由如下内容组成：

- 事件名称

可以理解为函数名，是事件的唯一表示。

- 事件参数

可以理解为函数的参数，是 JSON 格式的结构体，包含若干参数。

设备可以随时向云平台上报事件，只需将事件消息发布到如下主题：

event_report/{username}

发布的事件消息格式如下：

```
{
  "method": "{name}",
  "params": {
    "key1": "{value1}",
    "key2": "{value2}"
  }
}
```

例如，工厂生产线上的 AI 视觉传感器捕捉到产品数量后，上报事件给云平台，那么消息可能是这样的：

```
{
  "method": "productFound",
  "params": {
    "count": 3,
    "location": "REGION.B43",
    "tags": "RED",
    "detail": ["100474", "100475", "100342"]
  }
}
```

设备上报事件的响应

如果我们想知道事件上报是否被云平台成功接收，可以订阅如下主题：

event_response/{username}

当设备上报事件后，便会通过该主题收到来自云平台的响应消息，如果云平台接收成功，响应消息如下：

```
{
  "error_code": 0,
  "error_msg": "OK",
  "ts": 1750234507181
}
```

提示

响应消息中的成功，只代表云平台成功收到了事件上报，并不代表对事件进行业务处理的结果。

云平台收到设备的事件上报后，如何处理事件呢？**规则引擎**便派上用场了，通过规则引擎，您可以将事件实时推送到第三方，或通过规则引擎的云函数实现告警策略，当然还可以实现对其它设备的联动控制。

注意

此响应消息默认不开启，需要时可在此处开启：[设备类型详情页](#) > [设置](#) > [云端响应](#)。

云端下发命令至设备

与事件相反，命令是由云平台向设备主动下发的一组消息，用来实时通知设备执行某个特定的功能。

设备在MQTT连接云平台后，确保订阅如下主题：

command_send/{username}

订阅成功后，设备便处于等待接收命令的状态。当云平台下发命令时，设备便会收到命令消息。

命令消息的格式也是基于 JSON 的结构体，如下：

```
{
  "method": "{name}",
  "params": {
    "key1": "{value1}",
    "key2": "{value2}"
  },
  "id": "{id}"
}
```

它和事件消息格式非常类似，只是多了一个 id。您可以认为它就是从云平台向设备的一个远程调用（RPC）。

云端下发命令至设备的响应

设备回复命令发布的主题如下：

command_reply/{username}

设备收到命令消息后，通过自身的代码逻辑实现特定的功能前，可以回复命令，也可以不回复命令。

云平台会自动接收回复命令，并和下发命令进行匹配，一次下发命令只能接收一次回复命令，它们的匹配正是通过使用一致的 *id* 来保证。

设备回复命令消息的格式如下：

```
{
  "method": "{name}",
  "params": {
    "key1": "{value1}",
    "key2": "{value2}"
  }
}
```

它和命令下发的消息内容完全一样。

设备上报自定义数据

详情浏览 [自定义数据流的 MQTT 主题](#)。

云端下发自定义数据至设备

详情浏览 [自定义数据流的 MQTT 主题](#)。

一级子设备MQTT接入协议

一级子设备MQTT接入协议

⚠ 注意

不推荐新接入设备采用。本协议仅为兼容老设备而保留，未来可能会被停用。

[子设备MQTT接入协议](#)完全兼容本协议，前者支持本平台的全新特性“支持多级子设备”，建议优先采用该协议。

对于仅有一级子设备的父设备与平台通信的场景，可使用以下MQTT主题：

主题概览

以下主题用于子设备通过网关与云端通讯：

消息类型	主题	发布/订阅
子设备上报上线通知	gateway_connect/{username}	网关发布
子设备上报下线通知	gateway_disconnect/{username}	网关发布
子设备上报属性	gateway_attributes/{username}	网关发布
子设备上报属性的响应	gateway_attributes_response/{username}	网关订阅
子设备获取云端属性	gateway_attributes_get/{username}	网关发布
子设备获取云端属性的响应	gateway_attributes_get_response/{username}	网关订阅
云端下发属性至子设备	gateway_attributes_push/{username}	网关订阅
子设备上报事件	gateway_event_report/{username}	网关发布
子设备上报事件的响应	gateway_event_response/{username}	网关订阅
云端下发命令至子设备	gateway_command_send/{username}	网关订阅
云端下发命令至子设备的响应	gateway_command_reply/{username}	网关发布

各主题的使用方法

下面我们逐个介绍各个主题的使用方法。

子设备上报上线通知

当网关确定子设备已连接时，可上报消息通知平台，网关发布主题如下：

`gateway_connect/{username}`

网关 MQTT 的所有消息都必须是 JSON 格式，如果发布的不是 JSON 格式，网关会被平台主动断开 MQTT 连接。

```
{
  "device": "SUB_DEVICE_ADDR",
  "product": "ProductKey OR ProductCode"
}
```

device: 子设备地址。

product: 非必含。EZtCloud 公共产品库 中, 产品的 产品Key 或 产品识别码 。当包含此参数时, EZtCloud 将检查该设备是否已被创建, 若否, 则自动创建该设备。

子设备上报下线通知

当网关确定子设备已断开时, 可上报消息通知平台, 网关发布主题如下:

gateway_disconnect/{username}

发布消息格式如下:

```
{
  "device": "SUB_DEVICE_ADDR"
}
```

device: 子设备地址。

子设备上报属性

当网关接收子设备的属性变化时, 向平台上报子设备属性主题如下:

gateway_attributes/{username}

消息格式:

```
{
  "SUB_DEVICE_ADDR1": {
    "ATTRIBUTES1": "VALUE1",
    "ATTRIBUTES2": "VALUE2"
  },
  "SUB_DEVICE_ADDR2": {
    "ATTRIBUTES1": "VALUE1",
    "ATTRIBUTES2": "VALUE2"
  }
}
```

网关支持一次上报多个子设备的属性。

SUB_DEVICE_ADDR1、SUB_DEVICE_ADDR2: 子设备地址。

举个例子, 网关上报多个子设备的属性, 如下:

```
{
  "1": {
    "battery": 100,
    "occupancy": false
  }
}
```

```
},
"2": {
  "battery": 98,
  "occupancy": true
}
}
```

这时，在控制台的子设备详情页，会实时更新显示设备属性的最新值。

子设备上报属性的响应

网关订阅这个主题，可以实时接收平台下发给子设备的属性消息。

gateway_attributes_response/{username}

接收到的消息格式如下：

```
{
  "error_code": 0,
  "error_msg": "OK",
  "ts": 12312314212
}
```

device：子设备地址。

网关通过自身的运行机制，将消息转发给子设备地址 为 SUB_DEVICE_ADDR1 的子设备，或者在转发前对数据包进行必要的处理。

注意

此响应消息默认不开启，需要时可在此处开启：设备类型详情页 > 设置 > 云端响应。

子设备获取云端属性

网关可以获取子设备在平台的属性，例如用来初始化子设备的运行状态。

发布主题如下：

gateway_attributes_get/{username}

发布的消息格式

```
{
  "device": "SUB_DEVICE_ADDR1",
  "get": {
    "keys": ["KEY1", "KEY2"]
  }
}
```

device：子设备地址。

keys：要读取的属性标识符数组，空数组 [] 标识读取所有属性。

子设备获取云端属性的响应

网关订阅以下主题，用来接收平台回复的子设备属性。

gateway_attributes_get_response/{username}

回复的消息格式

```
{
  "device": "SUB_DEVICE_ADDR1",
  "error_code": 0,
  "error_msg": "",
  "ts": 12312314212,
  "attributes": {
    "KEY1": "VALUE1",
    "KEY1": "VALUE2"
  }
}
```

device：子设备地址。

response：读取子设备属性的回复内容。

云端下发属性至子设备

除了子设备主动获取属性以外，我们也需要让子设备可以实时接收云平台下发的属性。

确保网关已订阅如下主题：

gateway_attributes_push/{username}

当云平台下发属性给子设备时，网关会通过以上订阅主题，收到 JSON 结构的消息，例如：

```
{
  "device": "SUB_DEVICE_ADDR1",
  "attributes": {
    "switch": false
  }
}
```

这个示例消息显然是通知子设备关闭某个开关，子设备通过自身的程序实现该功能后，可以接着向云平台上报属性或上报事件，让云平台得到确认。

子设备上报事件

在前边的介绍中，我们已经知道，事件通常用于设备向平台发送一些通知或请求，但不希望将相关参数记录到设备属性中。

网关同样可以上报子设备的事件，并触发平台对子设备设置的事件规则。

例如，网关上报子设备请求 OTA 固件版本检查的事件。

发布主题如下：

gateway_event_report/{username}

上报的消息格式

```
{
  "device": "SUB_DEVICE_ADDR1",
  "event": {
    "method": "EVENT_IDENTIFIER",
    "params": {
      "PARAM1": "VALUE1",
      "PARAM2": "VALUE2"
    },
    "id": 1000
  }
}
```

device：子设备地址。

event：事件消息体，与直连设备事件上报的格式相同。

子设备上报事件的响应

如果我们想知道事件上报是否被云平台成功接收，可以订阅如下主题：

gateway_event_response/{username}

当网关上报子设备事件后，便会通过该主题收到来自云端的响应消息，如果云端接收成功，响应消息如下：

```
{
  "error_code": 0,
  "error_msg": "OK",
  "ts": 12312314212,
  "device": "SUB_DEVICE_ADDR1"
}
```

提示

响应消息中的成功，只代表云平台成功收到了事件上报，并不代表对事件进行业务处理的结果。

云平台收到子设备的事件上报后，如何处理事件呢？**规则引擎**便派上用场了，通过规则引擎，您可以将事件实时推送到第三方，或通过规则引擎的云函数实现告警策略，当然还可以实现对其它设备的联动控制。

注意

此响应消息默认不开启，需要时可在此处开启：设备类型详情页 > 设置 > 云端响应。

云端下发命令至子设备

订阅主题如下：

gateway_command_send/{username}

接收到的消息格式如下：

```
{
  "device": "SUB_DEVICE_ADDR1",
```

```
"command": {
  "method": "COMMAND_IDENTIFIER",
  "params": {
    "PARAM1": "VALUE1",
    "PARAM2": "VALUE2"
  },
  "id": 1000
}
```

device: 子设备地址。

command: 命令消息体，与直连设备下发命令消息的格式相同。

云端下发命令至子设备的响应

子设备收到命令消息后，在设备端实现特定的功能前，可以回复命令，也可以不回复命令。

云端会自动接收回复命令，并和下发命令进行匹配，在控制台的设备命令历史中，您可以看到一组包含下发和回复的命令日志。一次下发命令只能接收一次回复命令，它们的匹配是通过使用一致的 <id> 来保证。

发布主题如下：

gateway_command_reply/{username}

消息格式如下：

```
{
  "device": "SUB_DEVICE_ADDR1",

  "command": {
    "method": "EVENT_IDENTIFIER",
    "params": {
      "PARAM1": "VALUE1",
      "PARAM2": "VALUE2"
    },
    "id": 1000
  }
}
```

它和命令下发的消息内容完全一样。

子设备MQTT接入协议

子设备MQTT接入协议

对于子设备通过父设备（如网关）与EZtCloud通信的场景，父设备可使用以下MQTT主题：

主题概览

以下主题用于子设备通过父设备与云端通讯：

消息类型	主题	发布/订阅
上报子设备上线通知	sub_connect/{username}	父设备发布
上报子设备下线通知	sub_disconnect/{username}	父设备发布
上报子设备属性	sub_attributes/{username}	父设备发布
上报子设备属性的响应	sub_attributes_response/{username}	父设备订阅
获取子设备云端属性	sub_attributes_get/{username}	父设备发布
获取子设备云端属性的响应	sub_attributes_get_response/{username}	父设备订阅
云端下发属性至子设备	sub_attributes_push/{username}	父设备订阅
上报子设备事件	sub_event_report/{username}	父设备发布
上报子设备事件的响应	sub_event_response/{username}	父设备订阅
云端下发命令至子设备	sub_command_send/{username}	父设备订阅
云端下发命令至子设备的响应	sub_command_reply/{username}	父设备发布

各主题的使用方法

下面我们逐个介绍各个主题的使用方法。

上报子设备上线通知

当父设备确定子设备已连接时，可上报消息通知平台，父设备发布主题如下：

sub_connect/{username}

```
{
  "device": ["SUB_DEVICE_ADDR", "SUB_SUB_DEVICE_ADDR"],
  "product": "ProductKey OR ProductCode"
}
```

device：必含。子设备地址。它是一个列表，其中第0个元素表示本级子设备的子设备地址，第1个元素表示再下1级子设备的子设备地址，第2个元素表示再下2级子设备的子设备地址，依次类推。它至少应包含一个元素。

product: 非必含。EZtCloud 公共产品库 中, 产品的 产品Key 或 产品识别码。当包含此参数时, EZtCloud 将检查该设备是否已被创建, 若否, 则自动创建该设备。

上报子设备下线通知

当父设备确定子设备已断开时, 可上报消息通知平台, 父设备发布主题如下:

sub_disconnect/{username}

发布消息格式如下:

```
{
  "device": ["SUB_DEVICE_ADDR", "SUB_SUB_DEVICE_ADDR"]
}
```

device: 必含。子设备地址。它是一个列表, 其中第0个元素表示本级子设备的子设备地址, 第1个元素表示再下1级子设备的子设备地址, 第2个元素表示再下2级子设备的子设备地址, 依次类推。它至少应包含一个元素。

上报子设备属性

当父设备接收到子设备的属性变化时, 向平台上报子设备属性主题如下:

sub_attributes/{username}

EZtCloud支持如下2种消息格式: [树形格式](#)、[平铺格式](#)。两者功能等效, 设备商可自行人选其一实现。

消息成功上发后, 在控制台的设备详情页, 会实时更新显示设备属性的最新值。

树形格式:

```
{
  "SUB_DEVICE_ADDR_1": {
    "attributes": {
      "ATTRIBUTES1": "VALUE1",
      "ATTRIBUTES2": "VALUE2"
    },
    // 属性时间戳, 可选。不提供则记录为服务器时间
    "ts": 12312314212,
    "sub": {
      "SUB_DEVICE_ADDR_1_1": {
        "attributes": {
          "ATTRIBUTES1": "VALUE1",
          "ATTRIBUTES2": "VALUE2"
        },
        // 属性时间戳, 可选。不提供则记录为服务器时间
        "ts": 12312314211,
        "sub": {
          // 可递归包含子设备
        }
      },
      "SUB_DEVICE_ADDR_1_2": {
        "attributes": {
          "ATTRIBUTES1": "VALUE1",
```

```

        "ATTRIBUTES2": "VALUE2"
    }
    // sub 可为空，也可不含该键
    // "sub": {}
}
},
"SUB_DEVICE_ADDR_2": {
    // attributes 可为空，也可不含该键
    // "attributes": {}
    "sub": {
        "SUB_DEVICE_ADDR_2_1": {
            "attributes": {
                "ATTRIBUTES1": "VALUE1",
                "ATTRIBUTES2": "VALUE2"
            }
        }
    }
}
}
}
}

```

任何设备支持一次上报多个子设备的属性。

SUB_DEVICE_ADDR1、SUB_DEVICE_ADDR2：子设备地址。

SUB_DEVICE_ADDR_1_1、SUB_DEVICE_ADDR_1_2：子设备SUB_DEVICE_ADDR1的子设备地址。

ATTRIBUTES1、ATTRIBUTES2

举例，某设备上报多个子设备的属性，如下：

```

{
  "1": {
    "attributes": {
      "battery": 100,
      "occupancy": false
    },
    "sub": {
      "2": {
        "attributes": {
          "battery": 98,
          "occupancy": true
        }
      }
    }
  }
}
}

```

其中，“1”是当前设备的直接子设备，而“2”是“1”的子设备。

平铺格式：

```
[
  {
    "device": ["SUB_DEVICE_ADDR_1"],
    "ts": 12312314212,
    "attributes": {
      "ATTRIBUTES1": "VALUE1",
      "ATTRIBUTES2": "VALUE2"
    }
  },
  {
    "device": ["SUB_DEVICE_ADDR_1", "SUB_DEVICE_ADDR_1_1"],
    "ts": 12312314211,
    "attributes": {
      "ATTRIBUTES1": "VALUE1",
      "ATTRIBUTES2": "VALUE2"
    }
  },
  {
    "device": ["SUB_DEVICE_ADDR_1", "SUB_DEVICE_ADDR_1_2"],
    "ts": 12312314211,
    "attributes": {
      "ATTRIBUTES1": "VALUE1",
      "ATTRIBUTES2": "VALUE2"
    }
  },
  {
    "device": ["SUB_DEVICE_ADDR_2", "SUB_DEVICE_ADDR_2_1"],
    "attributes": {
      "ATTRIBUTES1": "VALUE1",
      "ATTRIBUTES2": "VALUE2"
    }
  }
]
```

举例，某设备上报多个子设备的属性，如下：

```
[
  {
    "device": ["1"],
    "attributes": {
      "battery": 100,
      "occupancy": false
    }
  },
  {
    "device": ["1", "2"],
    "attributes": {
      "battery": 98,
      "occupancy": true
    }
  }
]
```

```
}  
]
```

提示

上述两种格式中的示例是完全等效的。

上报子设备属性的响应

作为父设备的[直接上网设备](#)订阅这个主题，可以实时接收到平台对[上报子设备属性](#)的反馈消息。

sub_attributes_response/{username}

接收到的消息格式如下：

```
{  
  "error_code": 0,  
  "error_msg": "OK",  
  "ts": 12312314212  
}
```

error_code：错误码。0表示成功，非0表示失败。

error_msg：错误信息。以字符串形式，对错误进行详细说明。

ts：云平台发送本消息时的时间戳。

注意

此响应消息默认不开启，需要时可在此处开启：[设备类型详情页](#) > [设置](#) > [云端响应](#)。

获取子设备云端属性

父设备可以获取子设备在平台的属性，如下为典型的使用本主题的场景：父设备初始化子设备时，从云平台获取状态或配置信息。

发布主题如下：

sub_attributes_get/{username}

发布的消息格式

```
{  
  "device": ["SUB_DEVICE_ADDR", "SUB_SUB_DEVICE_ADDR"],  
  "get": {  
    "keys": ["KEY1", "KEY2"]  
  }  
}
```

device：必含。子设备地址。它是一个列表，其中第0个元素表示本级子设备的子设备地址，第1个元素表示再下1级子设备的子设备地址，第2个元素表示再下2级子设备的子设备地址，依次类推。它至少应包含一个元素。

keys：要读取的属性标识符数组，空数组 [] 标识读取所有属性。

获取子设备云端属性的响应

父设备订阅以下主题，以接收平台回复的子设备属性。

sub_attributes_get_response/{username}

回复的消息格式

```
{
  "device": ["SUB_DEVICE_ADDR", "SUB_SUB_DEVICE_ADDR"],
  "error_code": 0,
  "error_msg": "",
  "ts": 12312314212,
  "attributes": {
    "KEY1": "VALUE1",
    "KEY1": "VALUE2"
  }
}
```

device: 子设备地址。它是一个列表，其中第0个元素表示本级子设备的子设备地址，第1个元素表示再下1级子设备的子设备地址，第2个元素表示再下2级子设备的子设备地址，依次类推。它至少应包含一个元素。

error_code: 错误码。0表示成功，非0表示失败。

error_msg: 错误信息。以字符串形式，对错误进行详细说明。

ts: 云平台发送本消息时的时间戳。

attributes: 读取到的子设备属性。

云端下发属性至子设备

除了子设备主动获取属性以外，我们也需要让子设备可以实时接收云平台下发的属性。

确保父设备已订阅如下主题：

sub_attributes_push/{username}

当云平台下发属性给子设备时，父设备会通过以上订阅主题，收到 JSON 结构的消息，例如：

```
{
  "device": ["SUB_DEVICE_ADDR", "SUB_SUB_DEVICE_ADDR"],
  "attributes": {
    "switch": false
  }
}
```

device: 子设备地址。它是一个列表，其中第0个元素表示本级子设备的子设备地址，第1个元素表示再下1级子设备的子设备地址，第2个元素表示再下2级子设备的子设备地址，依次类推。它至少应包含一个元素。

attributes: 读取到的子设备属性。

这个示例消息显然是通知子设备关闭某个开关，子设备通过自身的程序实现该功能后，可以接着向云平台上报属性或上报事件，让云平台得到确认。

上报子设备事件

在前边的介绍中，我们已经知道，事件通常用于设备向平台发送一些通知或请求，但不希望将相关参数记录到设备属性中。

父设备同样可以上报子设备的事件，并触发平台对子设备设置的事件规则。

例如，父设备上报子设备请求 OTA 固件版本检查的事件。

发布主题如下：

sub_event_report/{username}

上报的消息格式

device：子设备地址。它是一个列表，其中第0个元素表示本级子设备的子设备地址，第1个元素表示再下1级子设备的子设备地址，第2个元素表示再下2级子设备的子设备地址，依次类推。它至少应包含一个元素。

event：事件消息体，与直连设备事件上报的格式相同。

上报子设备事件的响应

如果我们想知道事件上报是否被云平台成功接收，可以订阅如下主题：

sub_event_response/{username}

当父设备上报子设备事件后，便会通过该主题收到来自云端的响应消息，如果云端接收成功，响应消息如下：

```
{
  "error_code": 0,
  "error_msg": "OK",
  "ts": 12312314212,
  "device": "SUB_DEVICE_ADDR1"
}
```

提示

响应消息中的成功，只代表云平台成功收到了事件上报，并不代表对事件进行业务处理的结果。

云平台收到子设备的事件上报后，如何处理事件呢？**规则引擎**便派上用场了，通过规则引擎，您可以将事件实时推送到第三方，或通过规则引擎的云函数实现告警策略，当然还可以实现对其它设备的联动控制。

注意

此响应消息默认不开启，需要时可在此处开启：设备类型详情页 > 设置 > 云端响应。

云端下发命令至子设备

订阅主题如下：

sub_command_send/{username}

接收到的消息格式如下：

```
{
  "device": "SUB_DEVICE_ADDR1",
  "command": {
    "method": "COMMAND_IDENTIFIER",
```

```
    "params": {
      "PARAM1": "VALUE1",
      "PARAM2": "VALUE2"
    },
    "id": 1000
  }
}
```

device: 子设备地址。

command: 命令消息体，与直连设备下发命令消息的格式相同。

云端下发命令至子设备的响应

子设备收到命令消息后，在设备端实现特定的功能前，可以回复命令，也可以不回复命令。

云端会自动接收回复命令，并和下发命令进行匹配，在控制台的设备命令历史中，您可以看到一组包含下发和回复的命令日志。一次下发命令只能接收一次回复命令，它们的匹配是通过使用一致的 id 来保证。

发布主题如下：

sub_command_reply/{username}

消息格式如下：

```
{
  "device": "SUB_DEVICE_ADDR1",

  "command": {
    "method": "EVENT_IDENTIFIER",
    "params": {
      "PARAM1": "VALUE1",
      "PARAM2": "VALUE2"
    },
    "id": 1000
  }
}
```

它和命令下发的消息内容完全一样。

设备TCP接入协议

设备TCP接入协议

平台支持设备直接通过TCP接入云平台，并支持 JSON、Text、HEX 格式的上下行消息，为设备通信提供了无限的可能性。

TCP接入点

在设备详情的【连接】页面中，可以获取设备专用的TCP接入点，设备必须支持域名解析，格式如下：

TCP 接入点

tcp://tcp.ertcloud.com

TCP 地址: tcp.ertcloud.com

TCP 端口: 28801

注册包:

TCP注册包

当设备和云平台成功建立TCP连接后，设备必须马上向云平台发送注册包，完成身份认证。若设备端在一定时间内未发送身份信息，云平台会自动断开设备的TCP连接。

在使用TCP透传方式的网关或DTU中，可以使用注册包连接到云平台。

⚠ 注意

当设备通过注册包完成身份认证，有其它设备通过相同身份信息连接云平台，会自动顶掉之前的设备连接。

TCP心跳包

当设备和云平台建立TCP连接并完成身份认证后，便可以相互收发消息。但是，如果相当长一段时间内没有消息通信，双方如何判断对方仍然在线呢？因为TCP 对于一些非正常的连接断开是无法侦测到的，比如设备断电、网线断掉等。

因此，对于消息通信间隔较长的应用场景，为了让双方尽早的知道连接是否已经断开，从而实现重连，就需要有TCP保活机制，这是通过设备定期发送心跳包来实现的。

然而，大多数物联网通信场景的数据上报间隔时间并不长，所以也可以起到保活的目的，心跳包不是必须的。

自定义数据流

平台提供的TCP接入方式，不像MQTT一样拥有一套内置的设备访问协议，而是需要对 **TCP通道所属的自定义数据流**，设置相应的消息规则，实现自定义数据和设备属性之间的解析和处理。

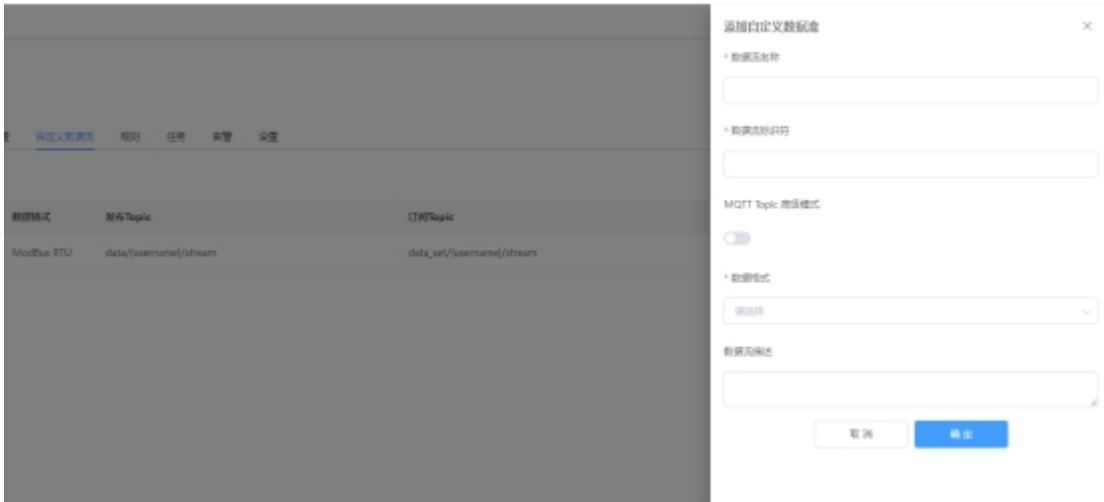
1. 创建设备类型

要开启自定义数据流，首先要创建设备类型。设备类型是用来统一定义设备的属性和功能。

在【设备类型】中可以创建设备类型，进入创建好的设备类型，在【关联设备】页面中，将设备添加到设备类型中，相当于为设备赋予了设备类型中定义的属性和功能。

1. 创建自定义数据流

在【自定义数据流】页面中，可以创建自定义数据流，如下：



自定义数据流自动生成了相应的MQTT发布主题和订阅主题，并允许设置数据格式，例如 Modbus RTU、自定义HEX、自定义ASCII、JSON 等。

1. 绑定TCP数据流

而对于TCP连接，我们只需要选择一个已创建的自定义数据流，进行绑定即可。

也就是说，被绑定的自定义数据流，同时也具备了TCP通道。



1. TCP自定义数据上报

完成上边的TCP绑定自定义数据流，设备端就可以通过TCP socket发送符合负载格式的数据，例如：类似属性消息结构的 JSON 消息，或者自定义的 HEX 消息。

平台收到TCP自定义数据上报后，可以创建规则引擎来对数据做各种解析和处理，平台提供了云函数等可编程方式，也可以将数据转发给第三方的 WebHook URL，进行其它专用的异步计算。

1. TCP自定义数据下发

要向已通过TCP连接到云平台的设备下发自定义数据，可以通过如下方式：

- 通过 **创建任务>自定义数据下发**
- 通过 **创建规则>属性下发>自定义数据下发函数**

设备通过DTU接入

设备通过DTU接入

DTU是Data Transfer Unit的简称，也就是数据传输单元。DTU是一种典型的网关产品，它的特点在于面向用户的快速配置能力，部分DTU还支持可编程。DTU将应用到更多的物联网场景中，成为一种开箱即用的硬件网关。

设备通过DTU接入平台时，并不需要编程知识，更多是基于配置界面来设置平台接入参数，以及和子设备的通信参数。

平台适配各类DTU产品，为物联网应用开发带来快速接入体验。

EZtCloud支持的DTU接入协议

- MQTT
- TCP

DTU通过MQTT方式接入云平台

对于支持MQTT的DTU产品，只需要简单几步就可以接入平台。

1.创建设备类型

- 创建一个设备类型，填写以下信息：
- 创建设备类型：选择自定义类型
- 类型名称：起个名字，例如：我的DTU
- 设备接入类型：直连设备
- 设备通讯方式：根据DUT联网方式
- 设备接入协议：选择Modbus RTU透传

2.创建自定义数据流

- 数据流名称：起个名称，例如：自定义数据流
- 数据流标识符：例如：modbus
- 数据格式：选择正确的数据流格式，例如：Modbus RTU格式

参考：[自定义数据流](#)。

3.创建设备

- 设备名称：起个设备名字，例如，我的DTU_01
- 设备类型：选择刚刚创建的设备类型，例如：我的DTU

4.获得设备MQTT连接参数

进入创建好的设备详情页，在【连接】页面中，找到用于MQTT连接的一些重要信息，包括：

- MQTT主机
- MQTT端口
- Username
- Password

5.配置DTU连接到云平台

不同的DTU产品支持不同的配置方式，通常有**上位机软件**和 **Web界面**等方式。

不论是哪种配置方式，都可以找到MQTT连接参数的配置界面，完成MQTT连接参数配置后，可以先保存DTU配置，这一步我们已经完成了DTU和云平台连接的身份认证配置。

6.使用MQTT自定义主题

每个自定义数据流会生成一组MQTT主题，包括一个发布主题和一个订阅主题，用来实现设备上报数据给平台，以及接收平台下发的数据，例如，我们创建一个标识符为modbus的自定义数据流，设置为支持 Modbus RTU 数据格式，那么平台会自动生成如下主题：

功能	类型	主题
上报数据	发布	data/modbus/{username}
下发数据	订阅	data_set/modbus/{username}

在实际使用中，以上这两个MQTT主题，可以实现平台向设备下发Modbus查询指令，设备向平台上报Modbus指令。平台通过设备类型中的Modbus寄存器设置或消息规则，来自动解析Modbus指令，提取相应的数据，自动生成设备属性。

7.配置DTU连接子设备

当我们完成以上配置后，如果DTU本身存在设备数据，例如DTU自身的版本信息，或者DTU本身支持IO输入，平台便可以开始接收DTU的真实数据上报。

除此之外，DTU的真正使命是转发其连接的子设备和云平台之间的消息。那么，就需要配置DTU和子设备的连接方式以及数据格式。

具体配置方法请参考DTU和子设备厂家所提供的的相关产品说明。

8.使用云平台辅助配置子设备

举个例子，我们在配置支持RS485/Modbus的传感器或IO控制器接入DTU时，由于我们事先已经配置好了DTU和云平台的连接，所以我们可以直接从云平台下发Modbus指令给传感器，完成从机地址修改等基本操作。

我们还可以用这种方式来远程调试传感器的数据上报准确性等，总之DTU为云平台和实际设备之间提供了通信桥梁，我们可以充分利用这种便利性。

9.使用平台定时任务

当DTU和子设备是主从关系时，我们需要DTU主动定时向子设备发送查询指令，有些DTU支持这样的功能，对于不支持的DTU，再或者我们需要远程灵活的调整查询策略，那么可以使用云平台提供的定时任务功能。

定时任务支持多种丰富的定时策略，例如：按间隔时间、按每日固定时间等。可执行的任务可以是下发一个 JSON 消息给设备，也可以是下发一个Modbus 查询指令，从而触发DTU连接的从机设备上报数据。

10.使用云平台规则引擎

同样以RS485/Modbus的传感器设备举例，为了快速解析Modbus协议，平台的规则引擎支持内置的Modbus解析规则，您可以在规则引擎中添加这样的操作，如下图：



规则配置非常简单，您只要略懂Modbus协议，并参考传感器设备的手册，便可以完成Modbus数据的提取。

11. 注意事项

- 如果DTU默认开启了一些预设主题的订阅，请将其关闭。否则会由于该主题不在平台支持的主题范围内，被平台限制MQTT接入。
- DTU发布数据的时间间隔不要低于30秒，否则可能会被平台断开。

DTU通过TCP透传方式接入云平台

TCP透传是DTU比较常见的一种平台连接方式，也是几乎所有DTU产品都支持的连接方式。这种方式让DTU作为一种透明转发的工作模式，将子设备发来的消息，直接转发给平台，同时也将平台下发的消息，转发给子设备。

总体来说，如果熟悉了前边的MQTT接入配置方法，那么TCP透传的配置方法基本类似，不同之处在于TCP接入点和设备身份认证的方式。

接下来我们介绍DTU TCP透传方式的接入步骤。

1. 创建设备类型

- 创建一个设备类型，填写以下信息：
- 创建设备类型：选择自定义类型
- 类型名称：起个名字，例如：我的DTU
- 设备接入类型：直连设备
- 设备通讯方式：根据DUT联网方式
- 设备接入协议：选择Modbus RTU透传

2. 创建自定义数据流

- 数据流名称：起个名称，例如：自定义数据流
- 数据流标识符：例如：modbus
- 数据格式：选择正确的数据流格式，例如：Modbus RTU格式
- **绑定TCP数据流**：选择自定义数据流，这相当于为该设备类型下的所有设备，开放了TCP接入通道。至于这里的Topic等信息，我们可以忽略，在使用TCP通道时没有实际作用。

3.创建设备

- 设备名称：起个设备名字，例如，我的DTU_01
- 设备类型：选择刚刚创建的设备类型，例如：我的DTU

4.获得设备TCP连接参数

进入创建好的设备详情页，在【链接】页面中，找到用于TCP连接的一些重要信息，包括：

- TCP地址
- TCP端口
- 注册包

5.配置DTU连接到平台

不同的DTU产品支持不同的配置方式，通常有上位机软件和 Web界面等方式。

不论是哪种配置方式，要配置的内容项基本相同，它们包括：TCP地址、TCP端口、注册包、心跳包(可选，用于设备TCP协议层保活)，完成以上配置后，可以先保存DTU配置。

到目前为止，我们在DTU端已经完成了TCP透传方式连接平台的所有配置。

6.数据发送和接受测试

为了测试DTU和平台是否已经通过TCP透传成功连接，可以在**设备详情页 > 调试**中，开启**调试**状态，用于查看设备发送和接收的消息。

常用的测试方法，例如：

用配置DTU的专用上位机软件，将DTU切换到透传模式，然后向DTU发送数据，在平台设备消息列表中刷新，检查是否会显示相应的消息。

同样在DTU的透传模式下，用其它任何串口调试工具，连接到DTU的从机端口，例如：RS485端口，模拟子设备发送消息给DTU，在云平台设备消息列表中刷新，检查是否会显示相应的消息。

7.上报DTU子设备的真实数据

最后，我们给DTU接上子设备，例如一个RS485 Modbus温湿度传感器，通过 Modbus RTU任务来定时下发Modbus查询指令，并通过Modbus RTU规则来自动解析Modbus消息，生成设备的属性数据。

设备消息调试

设备消息调试

平台为您提供了快捷的设备消息调试界面，可以帮助您：

- 实时监视设备和云平台之间双向传输的所有消息，支持 JSON、Plaintext、HEX 等格式。
- 查看消息格式错误的提示信息。
- 向设备下发消息，包括：下发属性、下发命令、下发自定义数据。

这些消息包括设备发送给云平台的消息，以及云平台下发给设备的消息。设备和云平台之间的所有互动都体现在消息中，所以，通过监视设备消息，可以帮助您快速定位和排查设备和云平台的通信是否符合预期。

开启消息日志

要查看这些消息，首先需要开启设备的消息日志，如下图：



每个消息可以点击右侧的图标，查看消息详情，如下图：



平台不仅支持JSON消息格式，还支持二进制和Plaintext 格式，例如 RS485/Modbus RTU消息是二进制格式。

查看错误信息

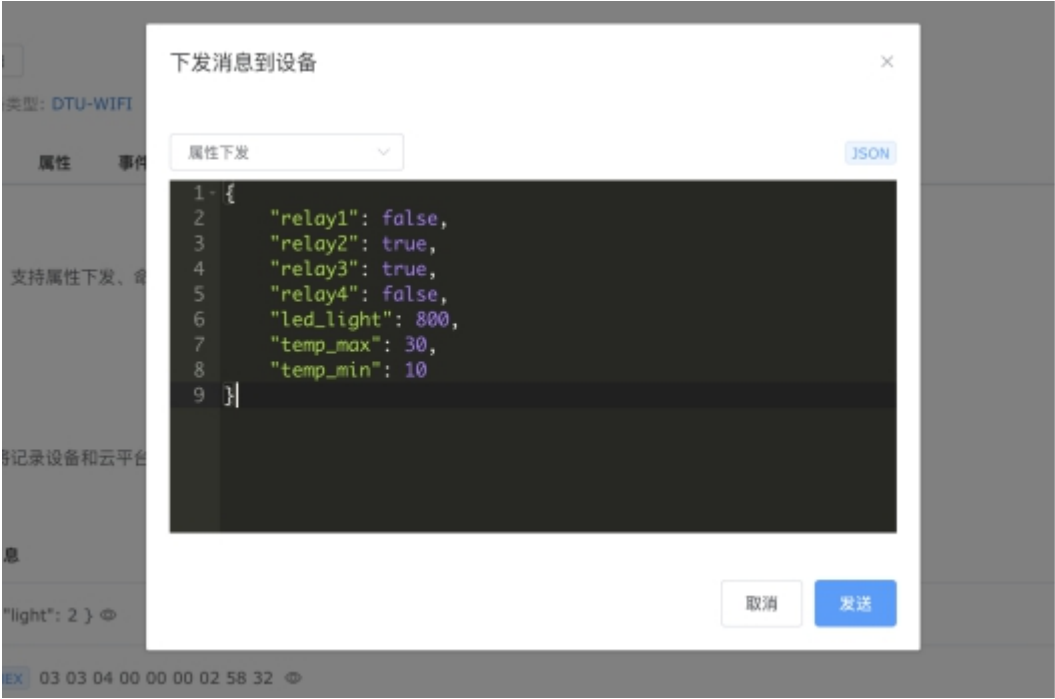
通过消息日志，还可以监视消息格式错误的原因，帮助您排查问题。

例如，如果您在设备类型中添加了属性定义，但设备上报的属性值不符合该属性定义时，云平台会拒绝接收消息，并产生错误信息。

向设备下发消息

设备调试界面中，可以快速为设备下发各类消息，可用于协议调试和设备故障排查。

1. 下发属性



1. 下发命令

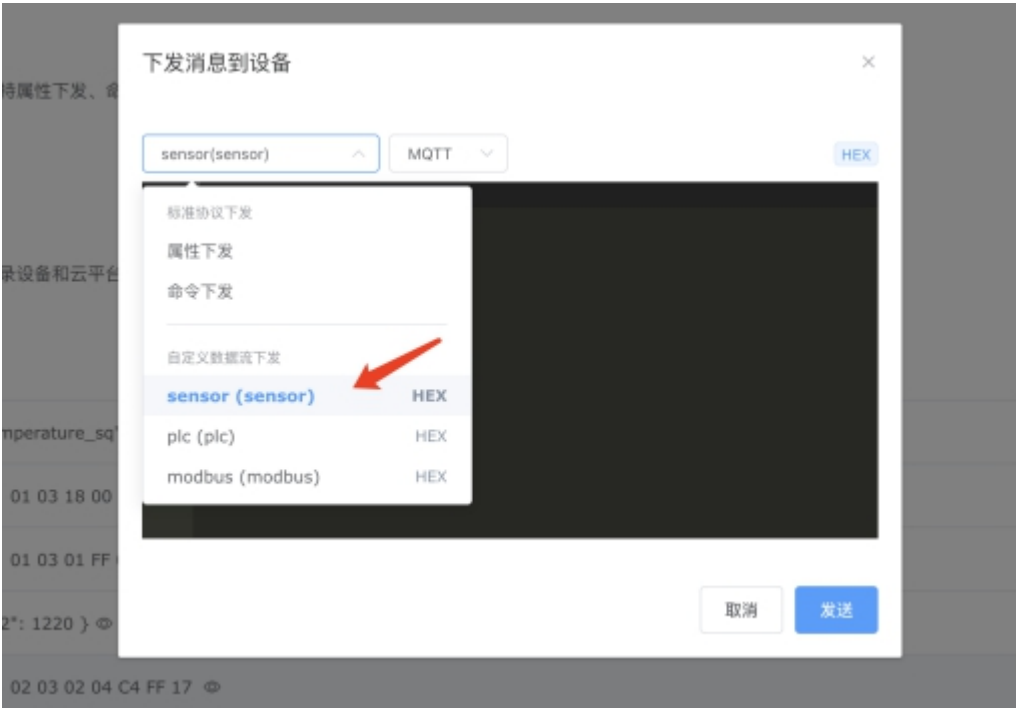


1. 下发自定义数据

在设备类型中添加自定义数据流

这样，我们在设备调试界面就可以选择自定义数据流，作为下发消息的通道。

这里我们为接入的DTU设备下发一个Modbus RTU查询指令，使用 HEX 消息格式。



DTU将指令透传到RS485串口总线，获得传感器的回复消息，同时解析为传感器属性 JSON，进入设备属性。

利用HEX 下发消息调试功能，还可以在设备安装部署时快捷执行一些一次性的指令，例如修改设备Modbus从机站号、对设备初始化配置等。

消息日志

开启消息日志后，云平台将记录设备和云平台之间的所有通信消息，包括用于排查问题的错误信息。



消息类型	消息
属性上报	{ "temperature_sq": 10.4, "temperature": 18.9, "humidity": 34.7, "temperature_id": 3.1 } ⓘ
自定义上报	HEX 01 03 18 00 68 00 BD 01 5B 00 1F F1 DC 41 97 0E FB 42 0B DE 8A 40 46 85 1E 41 27 75 C3 ⓘ
自定义下发	HEX 01 03 01 FF 00 0C 74 03 ⓘ

创建设备类型

创建设备类型

选择创建方式

创建设备类型时，有两种方式可以选择：

- 自定义：创建一个没有任何功能定义的设备类型，您需要根据设备需求来自行添加属性、事件、命令等功能定义，以及设置规则和任务等。
- 公共产品：从EZtCloud[公共产品库](#)中进行选择。如果您购买了已兼容厂商的硬件设备，可以通过这里的模板来**快速**初始化一个具有完整功能定义的设备类型，直接创建设备即可。

设备接入类型

表示设备通过哪种方式接入云平台，有以下三种类型：

- 直连设备：设备自带网络模组，例如：WiFi模组、4G模组等，可直接通过互联网接入云平台。
- 网关子设备：设备无法直接连接物联网，而需要通过网关来间接接入云平台。
- 网关：独立的网关设备，充当网关子设备和平台的桥梁，实现各类协议解析和消息转发。

选择通信方式

设备通信方式主要指设备连接物理层协议，例如：WiFi、2G/3G/4G/5G、BLE（低功耗蓝牙）、Zigbee、LoRa、RS485、以太网等，选择通信方式并不影响正常运行。

选择接入协议

1. 设备接入协议

对于直连设备和网关子设备，可选择以下接入协议：

- EZtCloud标准接入协议：作为设备接入的默认协议，基于MQTT协议，通过属性、事件、命令等消息，来实现设备和平台之间上报和下发的双向通信。参考：[设备MQTT接入协议](#)。
- Modbus RTU透传：设备和平台之间直接传输Modbus RTU消息报文，采用自定义数据流作为传输通道，支持MQTT或TCP接入方式。
- 其它：通过自定义数据流和规则引擎，可支持更多基于MQTT或TCP的自定义应用层协议。
- 网关接入协议

对于网关类型的设备，除了可以使用网关设备本身的接入协议，还可以使用网关协议，为子设备和平台提供间接的消息通信。

- EZtCloud标准网关协议：详情请浏览[一级子设备MQTT接入协议](#)。
- Modbus RTU云网关：用于Modbus RTU透传接入的DTU，通过对Modbus消息报文的处理，将DTU变为网关，将DTU的从机子设备映射到平台上的独立设备，便于各种场景对设备的灵活管理需要。
- 自定义：通过自定义数据流和规则引擎，实现更多自定义的网关接入协议。

关联设备

设备类型中可以绑定多个现有设备，也可以将设备从设备类型中移除，这些操作对设备都是实时生效。

功能定义

功能定义

功能定义是通过建立一套模型，来描述不同设备的能力，包括设备的属性、事件和命令，从而为设备接入和应用开发建立一套成熟的规范。功能定义对设备类型中的所有设备都有效。

对于熟悉面向对象编程的开发者而言，功能定义更像是给设备定义了一个类，其中约定了有哪些成员变量，以及变量的数据类型，另外还约定了一些特殊的方法。

属性

属性一般是设备自身具备的某种状态，比如温度值、开关状态、配置参数。

属性可以由设备上报到平台，或从平台下发到设备。平台会自动保存属性的当前值和历史值。

设备可拥有多个属性，例如：

```
{
  "temperature": 18.99,
  "humidity": 34.97,
  "pressure": 100390,
  "luminosity": 120,
  "pm10": 13,
  "pm2_5": 13,
  "relay1": false,
  "relay2": false,
  "version": "1.2.1"
}
```

1) 属性名称

用于给属性起一个便于区分的名称，支持中文，不出现在属性消息中。

在一些设备面板界面的属性当前值和时序图表中，属性名称会自动代替属性标识符，便于浏览。

2) 属性标识符

属性标识符中只能包含：英文大小写字母、数字、下划线、横线。

3) 属性类型

设备上报

这类属性只能由设备上报到云平台。支持的方式包括：

- 属性上报
- 规则引擎中的属性上报预处理
- 规则引擎中的自定义数据上报

设备上报属性的例子：

```
// 温度属性上报
{ "temperature": 28.4 }
```

云端下发

这类只能由云平台下发到设备的属性。

云端下发属性的例子：

```
// 电源开关控制
{ "power": false }
```

双向互发

这类属性既可以由设备上报，也可以由平台下发到设备，也就是可以同时被设备和平台改写。

双向互发属性的例子：

```
// 灯泡开关状态
{ "light": false }
```

云端私有

这类属性只能通过平台更新，不会下发到设备，也无法由设备上报到平台。

云端私有属性的例子：

```
// 供云端应用使用的温度阈值
{ "threshold": [0, 50] }
```

4) 数据类型

通过对属性设置数据类型，平台会自动验证属性的数据类型是否合法，对于不符合数据类型或不在设定范围内的情况，会自动忽略处理。

平台支持以下数据类型：

Number(数值)

任意数值，不区分整数和浮点数，例如：-12345、0、12.345、12345

附加选项：

- 单位
- 精度
- 最小值
- 最大值

当设置数值属性的最大值或最小值后，如果上报或下发的数值属性不符合该条件，平台会拒绝处理，您可以在设备调试中查看详细错误提示，便于修改合理的限制条件。

Text(文本)

Plaintext 字符串，例如：auto

例如，设备上报文本型的属性格式：

```
{ "mode": "auto" }
```

Switch(开关量)

Switch开关量用于表示两种状态，例如LED的开/关，继电器的闭合/断开。

开关量使用布尔类型，如：true/false

为了支持各种硬件设备的开关量表达方式，开关量允许在设备和平台之间的消息JSON中使用多种开关值，如下：

- 布尔类型:true/false
- 整数类型:1/0
- 文本类型:ON/OFF

附加选项：

- ON文字
- OFF文字

例如，设备上报开关量型的属性格式：

```
{
  "switch1": false,
  "switch2": true
}
```

Enum(枚举)

Enum类型的属性，也是一种文本类型，区别在于必须在限定的枚举值中取值。

例如，设备上报枚举型的属性格式：

```
{ "mode": "1" }
```

Object(键值对)

```
{
  "key1": "value1",
  "key2": "value2",
  "key3": "value3"
}
```

value表示已支持的任意基本数据类型。

例如，设备上报键值对型的属性格式：

```
{
  "data": {
    "temperature": 31.2,
    "switch": true,
    "mode": "1"
  }
}
```

List(列表)

```
["value1", "value2", "value3"]
```

value 表示已支持的任意基本数据类型。

例如，设备上报列表型的属性格式：

```
{ "data": [1, 3, 5] }
```

事件

事件一般是设备向平台的通知或请求，可携带自定义的参数，平台或应用端需要及时处理。比如故障通知、任务进度通知。

1)事件名称

用于给事件起一个便于区分的名称，支持中文，不出现在事件消息中。

2)事件标识符

事件标识符用在事件消息中，是method 字段的值。

事件标识符中只能包含：英文大小写字母、数字、下划线、横线。

3)事件参数

事件消息中的 params 字段可携带事件参数，事件参数是一个 键值对集合，例如下边这个事件消息的参数：

```
{
  "method": "alarm",
  "params": {
    "code": "1001",
    "module": "pressure-sensor"
  }
}
```

在设备类型中创建事件时，如果定义了参数，云平台会严格验证事件消息中该参数的数据类型。

命令

命令是从平台向设备下发的通知或请求，可携带参数。比如下发OTA升级、重启等命令。

设备端在收到平台下发的命令消息后，根据命令标识符和命令参数，进行相应的处理。

设备端处理结束后，可以向平台上报命令回应消息，用于发送处理结果。但命令回应消息不是必须的，设备端也可以用属性上报或事件上报，让平台或应用端获得需要的信息即可。当然，对于有些下发的命令，设备端处理完成后，也可以不上报任何消息，这取决于下发命令的需求和目的。

1)命令名称

用于给命令起一个便于区分的名称，支持中文，不出现在命令消息中。

2)命令标识符

命令标识符用在命令消息中，是method字段的值。

命令标识符中只能包含：英文大小写字母、数字、下划线、横线。

3)命令参数

命令消息中的params字段可携带命令参数，命令参数是一个 键值对集合，例如下边这个命令消息的参数：

```
{
  "method": "restart",
  "params": {
    "delay": 30
  }
}
```

在设备类型中创建命令时，如果定义了命令参数，平台会严格验证命令消息中该参数的数据类型。

4)回应参数

命令回应消息不是必须的，如果发送它，也可以携带回应参数。

命令回应消息和命令消息的格式一模一样，区别在于只能从设备向平台发送。例如：

```
{
  "method": "restart",
  "params": {
    "result": true
  }
}
```

在设备类型中创建命令时，如果定义了回应参数，平台会严格验证命令回应消息中该参数的数据类型。

5)命令实例

本系统支持对设备类型或设备创建命令实例。

命令实例中不仅明确了命令标识符、命令参数，还明确了各命令参数的值。

创建命令实例可以帮助用户一键快速下发具有复杂参数组合的命令。

对某个设备类型创建命令实例后，该设备类型下的所有设备都可方便的使用该命令实例。

自定义数据流

自定义数据流

自定义数据流是对EZtCloud标准通信协议的补充和扩展，用来支持更多类型的设备接入。

EZtCloud为设备接入提供了简单易用的MQTT接入协议，并在MQTT的基础上提供了一套完整的应用规范，包括规定MQTT主题和消息格式。

这些内置的规范有助于设备端的开发过程变得更加简单。但是对于另一些情况，您可能需要使用自定义数据流。比如：

- 由于项目开发的某些需求，需要使用自定义的MQTT主题，或者需要使用自定义的消息格式
- 硬件和云平台的通信需要使用二进制消息。例如：Modbus RTU报文消息
- 需要使用一些现成的设备，只支持TCP网络通信。例如：仅支持TCP连接的DTU设备。

这时，可以为设备类型创建自定义数据流，从而使该设备类型下的所有设备都支持该自定义数据流。

创建自定义数据流

进入 **设备类型** > **设备类型详情** > **自定义数据流** > **创建自定义数据流**，即可创建自定义数据流。

- 数据流名称：支持中文
- 数据流标识符：用在自定义数据流的MQTT主题中，仅支持：英文大小写字母、数字、下划线、横线
- 数据格式：选择自定义数据流中传输的消息格式

数据格式

选择正确的数据格式，用于云平台对自定义数据流传输的消息进行合法性验证。

消息格式可以随时修改，支持以下数据格式：

Modbus RTU

设备和云平台之间传输的是Modbus RTU二进制消息，通常用在DTU连接云平台，将Modbus子设备的报文直接透传到云平台。

例如：01 03 0A 00 16 00 53 00 38 00 08 00 03 C1 D5

Binary 二进制消息

任意自定义的二进制消息。

通常使用自定义二进制消息的场景例如：

- 设备现有通信协议仅支持二进制消息，例如一些工业设备。
- 需要严格限制通信数据流量，用二进制消息尽可能减少消息长度。

JSON 格式消息

自定义的JSON消息格式，云平台不对消息格式做任何合法性检查。

Plaintext 文本消息

自定义的 Plaintext 消息格式，云平台不对消息格式做任何合法性检查。

自定义数据流的MQTT主题

每个自定义数据流都拥有一组预定义好的MQTT主题，如下：

消息类型	主题
发布主题	data/{username}/{attributes}
订阅主题	data_set/{username}/{attributes}

自定义数据流在云平台处理

设备通过自定义数据流和云平台建立消息通信，云平台可以通过规则对自定义数据流进行各种处理，比如解析Modbus RTU消息并生成预设的属性值。云平台还可以通过规则将自定义数据流转发给第三方应用或接口。

例如：使用Modbus RTU数据格式的自定义数据流，设备向云平台上报了消息：01 03 0A 00 16 00 53 00 38 00 08 00 03 C1 D5

通过在规则中启用Modbus RTU解析规则，云平台在收到以上消息后，会立即自动解析，生成以下设备属性：

```
{
  "temperature": 22,
  "humidity": 83,
  "hs_temperature": 56,
  "cs_temperature": 8,
  "device_status": 3
}
```

绑定TCP数据流

自定义数据流除了本身具备MQTT主题，还可以开放TCP接入。设备的TCP接入方式实际上是复用了自定义数据流，当创建自定义数据流后，便可以将某个自定义数据流绑定为TCP数据流。

一个设备类型中，只能有一个自定义数据流可以绑定到TCP数据流。绑定后，设备类型下的所有设备便支持TCP接入，您可以在[设备详情页 > 连接](#)，查看TCP接入点的主机名和端口。

Modbus寄存器设置

Modbus寄存器设置

Modbus寄存器设置可以理解为专门为Modbus协议类设备开发的一个快捷、方便、易用的规则。

当设备类型的接入协议选择 Modbus RTU透传时，便可以在设备类型详情页中显示 Modbus配置，用于将设备属性和Modbus寄存器进行绑定，实现设备属性和 Modbus消息之间的自动转换，例如：

- 当云平台对设备下发一个开关量属性，由于这个属性绑定了一个设备上的读写类IO寄存器，那么实际下发到设备的消息，是一个 Modbus写入寄存器的消息报文。
- 当云平台通过任务向设备下发Modbus查询消息后，设备立即回应Modbus 响应消息，这时候云平台通过配置好的 Modbus 寄存器，将自动解析 Modbus 消息，生成相应的属性值。

属性智能转换

这是一个全局的开关，开启后，平台将使用Modbus寄存器地址表，自动解析设备上报的Modbus消息，并将属性下发转换为 Modbus消息。



选择自定义数据流

Modbus配置中需要设置自定义数据流，需要和Modbus透传用到的自定义数据流标识符保持一致。

推送方式支持MQTT和TCP，请选择设备实际接入的方式。

设置IO寄存器

IO寄存器的设置比较简单，点击**编辑IO寄存器**，IO寄存器处于可编辑状态，点击**添加一个寄存器或者添加多个寄存器**。

设置

打开6

子设备地址

5

只读

读写

删除

打开7

子设备地址

6

只读

读写

删除

打开8

子设备地址

7

只读

读写

删除

添加一个寄存器

添加多个寄存器

数据寄存器

编辑数据寄存器

属性	从机地址	寄存器地址	读写类型	数据类型	字节
----	------	-------	------	------	----

其中的属性，需要先在功能定义中创建，然后在添加Modbus寄存器时直接选择即可。

⚠ 注意

IO寄存器必须绑定到开关量（Boolean）属性上，且仅支持设备上报和设备云端共享两种类型的属性。

设置数据寄存器

新增数据寄存器流程与IO寄存器一致。

⚠ 注意

数据寄存器必须绑定到数值型（Number）属性上，且仅支持设备上报和双向传输两种类型的属性。

数据寄存器数据类型

平台对Modbus数值寄存器进行消息生成和自动解析时，支持以下数据类型：

数据类型	寄存器个数	字节数	位数	字节顺序
16位整数	1	2	16	AB/BA
16位无符号整数	1	2	16	AB/BA
32位整数	2	4	32	ABCD/BADC/CDAB/DCBA
32位无符号整数	2	4	32	ABCD/BADC/CDAB/DCBA
32位浮点数	2	4	32	ABCD/BADC/CDAB/DCBA

寄存器读写类型

在配置寄存器时，需要为寄存器选择准确的读写类型。

1)对于 **IO寄存器**，读写类型表示以下含义：

读写类型	寄存器类型	读功能码	写功能码
只读	读输入状态寄存器	02	不支持
读写	线圈状态寄存器	01	05

例如：

- 对继电器属性（开关量类型）设置为读写类型。

下发属性时，会自动生成05功能码的Modbus控制指令。

通过建立任务，下发01功能码的Modbus查询指令，获得继电器的最新状态。

- 对DI属性（开关量类型）设置为只读类型。

通过建立任务，下发02功能码的Modbus查询指令，获得开关量输入的最新状态。

若需要实时获取开关量输入状态，可使用带有边缘轮询功能的网关，当检测到开关量输入变化时，实时向云平台上报 JSON 属性。

2)对于 **数据寄存器**，读写类型表示以下含义：

读写类型	寄存器类型	读功能码	写功能码
只读	输入寄存器	04	不支持
读写	保持寄存器	03	06

创建Modbus下发任务

为了让平台自动读取设备数据，需要创建 Modbus下发类任务，例如：



有了配置好的Modbus寄存器，平台将设备上报的Modbus RTU报文自动转换成绑定的属性值，在控制台的设备详情页可以看到属性数据正在实时更新，通过任务的定时功能，我们可以实现固定间隔时间的读取设备数据。

属性下发转换为Modbus消息

Modbus 的另一个重要用途是对设备寄存器的写入，当我们在控制台对设备下发一个开关量属性时，设备会收到 Modbus 写寄存器的消息。数值型属性的下发也同样支持。

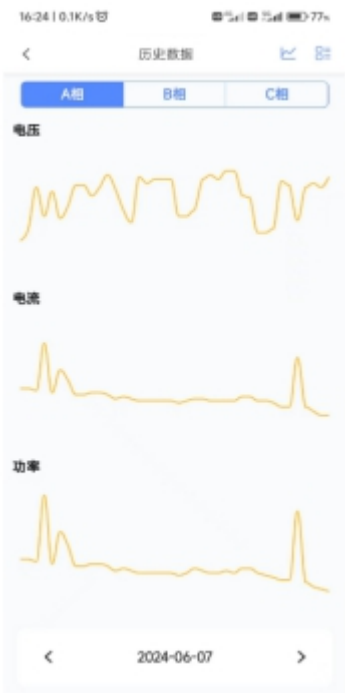
个性化UI

个性化UI

我们为一些广泛应用的设备类型定义个性化UI，以更好的展示和统计数据。个性化UI通过识别**属性标识符**来确定数据展示的位置。**固定属性标识符**，在您创建功能定义时需严格保持一致。

电气仪表





17:33 | 34.2K/s | 5.4V 5.4V 69%

< 历史数据 >

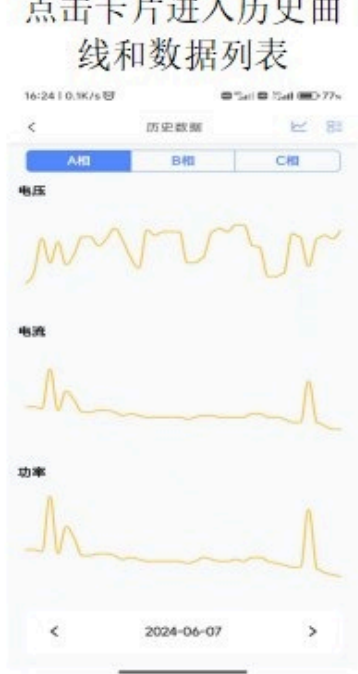
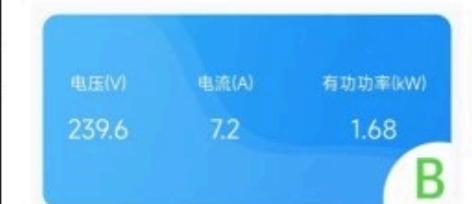
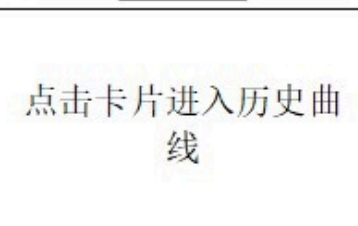
A相 B相 C相

时间	电压	电流	功率
17:33:12	240.7 V	2.3 A	0.48 kW
17:31:12	240.1 V	2.4 A	0.5 kW
17:29:12	241 V	2.4 A	0.5 kW
17:27:12	240.9 V	2.4 A	0.5 kW
17:25:12	242.1 V	2.4 A	0.5 kW
17:23:12	242.1 V	2.4 A	0.5 kW
17:21:12	240.9 V	2.4 A	0.5 kW
17:19:12	239.2 V	2.4 A	0.5 kW
17:17:12	238.5 V	2.3 A	0.48 kW
17:15:12	232.6 V	11.4 A	2.62 kW
17:13:12	234.1 V	11.6 A	2.7 kW

< 2024-06-07 >



固定属性标识符列表,区分大小写:

标识符	功能定义	显示位置	历史数据	
<u>pi</u>	输入有功电能			
<u>ua</u>	A 相电压			
<u>ia</u>	A 相电流			
<u>pa</u>	A 相功率			
<u>ub</u>	B 相电压			
<u>ib</u>	B 相电流			
<u>pb</u>	B 相功率			
<u>uc</u>	C 相电压			
<u>ic</u>	C 相电流			
<u>pc</u>	C 相功率			
<u>p</u>	总有功功率			

非固定属性标识符，显示在主界面列表，非必须按照我们规定的标识符，但为了尽可能使标识符规范化，还是将我们采用的标识符列在下表，供您参考：

标识符	功能定义	显示位置	历史数据
uab	AB线电压	设备主界面列表展示	
ubc	BC线电压	设备主界面列表展示	
uca	CA线电压	设备主界面列表展示	
u0	零序电压	设备主界面列表展示	
i0	零序电流	设备主界面列表展示	
ma	剩余电流	设备主界面列表展示	
uunb	电压不平衡度	设备主界面列表展示	
iunb	电流不平衡度	设备主界面列表展示	
hz	频率	设备主界面列表展示	

标识符	功能定义	显示位置	历史数据
qa	A相无功功率	设备主界面列表展示	
qb	B相无功功率	设备主界面列表展示	
qc	C相无功功率	设备主界面列表展示	
q	总无功功率	设备主界面列表展示	
sa	A相视在功率	设备主界面列表展示	
sb	B相视在功率	设备主界面列表展示	
sc	C相视在功率	主界面列表展示	
s	总视在功率	主界面列表展示	
pfa	A相功率因数	主界面列表展示	
pfb	B相功率因数	主界面列表展示	
pfc	C相功率因数	主界面列表展示	
pf	总功率因数	主界面列表展示	
t01	温度1	主界面列表展示	
t02	温度2	主界面列表展示	
t03	温度3	主界面列表展示	
t04	温度4	主界面列表展示	
epe	输出有功电能	主界面列表展示	
eqc	输出无功电能	主界面列表展示	
pjr	输入无功电能	主界面列表展示	
thdu_ua	A相电压总谐波	主界面列表展示	
thdu_ub	B相电压总谐波	主界面列表展示	
thdu_uc	C相电压总谐波	主界面列表展示	
thdi_ia	A相电流总谐波	主界面列表展示	
thdi_ib	B相电流总谐波	主界面列表展示	
thdi_ic	C相电流总谐波	主界面列表展示	
.....			更多

注1：对于**属性下发**和**双向传输**会全部展示在“设置”页面

创建规则引擎

创建规则引擎

什么是规则

规则引擎在EZtCloud中发挥着重要的作用，它可以按照设定的条件，对设备和平台之间传递的消息进行各种计算、处理、流转等操作。

例如用规则可以实现以下场景：

- 对设备上报的原始数据进行处理，生成我们需要的最终数据，保存在设备属性中。
- 将设备上报的属性，直接转发到第三方MQTT服务器。
- 将设备上报的事件，直接转发到第三方HTTP API接口。
- 当设备上报属性达到某个条件时，向另一个设备下发属性或命令。
- 当设备上报事件时，向另一个设备下发属性或命令。
- 对设备上报的Modbus RTU二进制消息进行解析，生成相应的设备属性。
- 对设备上报的非标准二进制协议，进行任意的解析，生成相应的设备属性。
- 向设备下发开关量属性时，自动生成设备能够识别的二进制报文。
- 向设备下发数值参数时，自动生成设备能识别的二进制报文。

创建规则

创建规则需要了解以下基本概念：

1) 规则类型

规则类型决定了规则的触发时间点，目前包括以下类型：

- 属性上报预处理：在云平台接收到设备端上报的属性后，且更新设备属性前，触发该类规则。
- 属性上报：在云平台正式更新设备属性后，触发该类规则。
- 事件上报：在云平台收到设备事件上报后，触发该类规则。
- 属性下发预处理：在云平台将要向设备端下发属性时，触发该类规则。
- 属性下发：在云平台向设备端下发属性后，触发该类规则。
- 自定义上报：在云平台收到设备自定义数据上报后，触发该类规则。

2) 设备来源类型

- 设备：将规则的作用范围限定在指定的一个或多个设备。
- 设备类型：将规则的作用范围限定在设备类型下的所有设备。

因此，如果只想对某个或某几个设备定义规则，则选择设备作为来源类型；如果希望对设备类型下的所有设备都定义同样的规则，则选择设备类型作为来源类型。

3) 属性条件

当规则类型是属性上报预处理或属性上报时，可以设置条件，条件设置不是必须的，当设置条件后，只有当上报的属性符合该条件时，才会触发该规则。

4) 事件标识符

当规则类型是事件上报时，需要设置事件标识符。只有当设备上报的事件符合时，才会触发该规则。

5) 自定义数据流标识符

当规则类型是自定义上报时，需要设置自定义数据流标识符。只有当设备上报的自定义数据符合时，才会触发该规则。

6) 操作

操作是规则中的重要组成部分，一个规则被触发后，实际上执行的是预先设定好的操作部分。

添加操作：每类操作都是一个独立的组件，在后边章节中，会针对不同类型的规则，详解介绍如何使用各类操作。

操作排序：一个规则中允许有多个操作，操作的顺序决定了执行的顺序。

对于属性上报预处理类型的规则，每个操作都是同步执行，多个操作是串行执行，也就是前一个操作生成的属性集合，是下一个操作的属性输入。

而对于其它类型的规则，操作大多是异步执行，因此操作的顺序对运行结果并没有直接影响。

规则状态

每个规则拥有独立的状态，分为：

- 已启用：规则可被触发。
- 已停用：规则被忽略。

与此同时，规则还可以对其作用范围内的每个设备单独设置是否有效。例如：一个规则属于设备类型，规则全局状态是已启用，但是对某个设备可以关闭规则，如下图：



规则统计

在**规则详情页** > **统计**以及**设备详情页**，都可以查看规则的调用次数统计。

规则日志

在**规则详情页** > **日志**中开启调试状态，便可以记录每次规则被触发的日志，便于查看规则是否被触发。

属性上报预处理

属性上报预处理

属性上报预处理规则

属性上报预处理规则的触发时间，是在云平台接收到硬件上报的属性，且在正式更新设备属性之前。适合用于对设备上报属性做必要的格式处理、二次计算、转化、合并等，总之可以进行任意的加工。

目前支持的操作如下：



预处理函数

提供了可编程的云函数，支持 Javascript 编程语言，如下：



预处理函数用法举例

属性上报

属性上报

属性上报规则

当云平台接收设备端的属性上报消息，并更新设备属性后，会触发该类规则。

属性上报规则目前支持以下操作：

添加操作

向当前设备下发属性

可实现属性上报触发自定义属性下发逻辑。

向当前设备下发命令

可实现属性上报触发自定义命令下发逻辑。

推送到外部MQTT

将设备上报的属性值转发到外部的 MQTT Broker，实现更多基于实时数据的应用。

推送到外部URL

将设备上报的属性值转发到外部的 URL，方便 Web 应用获取设备实时数据。

向当前网关上报子设备属性

将属性上报转换为网关子设备属性上报。

云函数

编写基于 Javascript 的云函数，调用内置函数库实现各类操作。

取消

向当前设备下发属性

操作用于向当前设备下发属性，实现设备联动功能。

操作配置

函数

```
1 function downAttr(report_attributes) {  
2     /**  
3      * report_attributes: 上报的属性对象，作为函数参数传入  
4      * push_attributes: 构造下发的属性对象，作为函数返回值，下发到硬件  
5      */  
6     var push_attributes = {  
7  
8     };  
9     return push_attributes;  
10 }
```

取消 确定

参数

- report_attributes: 是设备上报的属性集合，作为参数传入函数。

返回值

- object 类型: 构造一个下发到目标设备的属性集合。
- null: 表示不下发属性到设备。

示例

例如，我们需要当设备上报开关量 di1 属性变为 ON 后，自动断开继电器 do1，可以编写如下的云函数：

```
function downAttr(report_attributes) {  
  /**  
   *report_attributes: 上报的属性对象，作为函数参数传入  
   *push_attributes: 构造下发的属性对象，作为函数返回值，下发到硬件  
   */  
  var push_attributes = {}; // 这里的 === 表示上报属性中必须存在 di1 且为 true  
  if (report_attributes.di1 === true) {  
    push_attributes.do1 = false;  
  }  
  return push_attributes;  
}
```

向当前设备下发命令

该操作用于向当前设备下发命令，实现设备联动功能。

函数

```
1 * function downCmd(report_attributes) {  
2 *   /**  
3 *     * report_attributes: 上报的属性对象，作为函数参数传入  
4 *     * command: 构造下发的命令对象，作为函数返回值，下发到硬件  
5 *     */  
6 *     var command = {  
7 *       method: 'command_identifier',  
8 *       params: {  
9 *         x: 1,  
10 *        y: 1  
11 *      }  
12 *     }  
13 *   }  
14 * }
```

取消

确定

参数

- report_attributes: 是设备上报的属性集合，作为参数传入函数。

返回值

- object 类型: 构造一个下发到目标设备的命令消息。

- null：表示不下发命令到设备。

推送到外部MQTT

该操作将设备上报的属性直接转发到第三方MQTT服务器，需要填写MQTT主机名、端口、身份验证信息等。

操作配置 ×

MQTT 服务器主机名/IP

MQTT 服务器端口

client ID

username

password

发布到 topic 🔍

SSL/TLS ☐

取消 确定

推送到外部URL

该操作将设备上报的属性直接转发到第三方 URL。

操作配置 ×

URL 🔍

取消 确定

向当前网关上报子设备属性

该操作将设备上报的属性转发到子设备，仅用于当前设备是网关类型的情况下。

参数

- report_attributes：是设备上报的属性集合，作为参数传入函数。

返回值

- object 类型：构造一个向网关上报子设备的消息，消息格式请参考：[上报子设备属性](#)。
- null：表示不转发消息到子设备。

函数

```
1* function reportSubAttr(report_attributes) {
2*   /**
3*    * report_attributes: 上报的属性对象，作为函数参数传入
4*    */
5*
6*   var data = {};
7*
8*   return data;
9* }
```

取消

确定

示例

```
function reportSubAttr(report_attributes) {
  /**
   *report_attributes: 上报的属性对象，作为函数参数传入
   */
  var data = {};
  if (report_attributes.devEUI) {
    data[report_attributes.devEUI] = report_attributes;
  }
  return data;
}
```

自定义云函数

自定义云函数操作可处理自定义逻辑，并支持一些内置函数，例如：

- 更新设备云端私有属性
- 给指定设备下发属性
- 给指定设备下发命令

事件上报

事件上报

事件上报规则

当云平台接收设备端的事件上报消息后，会触发该类规则。

添加操作 ✕



OTA 升级检查

通过设备事件来实现设备 OTA 升级检查，自动关联 OTA 版本库，将需要升级的固件信息下发给设备。



推送到外部MQTT

将设备上报的事件消息转发到外部的 MQTT Broker，实现更多基于实时数据的应用。



推送到外部URL

将设备上报的事件消息转发到外部的 URL，方便 Web 应用获取设备实时数据。



向其它设备下发属性

可以向项目中的其它设备下发属性值，实现便捷的设备联动控制。



向其它设备下发命令

可以向项目中的其它设备下发命令，实现便捷的设备联动控制。



云函数

可以自定义编写基于 Javascript 的云函数，调用内置函数库实现各类操作。

取消

OTA检查升级

用于设备OTA固件升级时，由设备请求检查固件新版本。详细说明请查看 [OTA固件升级](#)

操作配置

×

版本属性

请填写属性标识符

模块

请选择

取消

确定

推送到外部MQTT

该操作将设备上报的事件消息直接转发到第三方MQTT服务器，需要填写 MQTT 主机名、端口、身份验证信息等。

操作配置

×

MQTT 服务器主机名/IP

MQTT 服务器端口

client ID

username

password

发布到 topic ?

SSL/TLS

取消

确定

推送到外部URL

该操作将设备上报的事件消息，通过HTTP请求转发到您设置的第三方URL。

操作配置

URL ?

取消 确定

请求格式

HTTP请求头

Content-Type: application/json

HTTP正文

采用 JSON 格式，数据示例如下：

```
{
  "device": {
    // 设备基础信息
    "id": "string",
    "name": "string",
    "device_key": "string"
  },
  "event": {
    // 设备上报的事件
    "method": "string",
    "params": {
      // ...
    },
    "id": number
  },
  // 当前 Unixstamp 时间戳
  "ts": number
}
```

云函数

操作配置

×

函数

```
1 function main(event) {
2   /**
3    * event: 传入事件对象，由硬件上报
4    */
5
6   return;
7 }
```

测试云函数 ^

取消

确定

除了各类专用的操作组件外，云平台面向企业客户提供通用的 云函数，支持比较大的自由度，可以自定义处理逻辑，并支持一些内置函数，例如：

- 更新设备云端私有属性
- 给指定设备下发属性
- 给指定设备下发命令

向其他设备下发命令

该操作用于向其它设备下发命令，实现设备联动功能。

设备来源类型为：设备

下发到设备

请选择设备

函数内容

```
1 function main(event) {  
2     /**  
3      * event:          传入事件对象，由硬件上报  
4      * push_attributes: 构造下发的属性对象，作为函数返回值，下发到硬件  
5      */  
6     var push_attributes = {  
7  
8     };  
9     return push_attributes;  
10 }
```

[测试云函数](#) ^

取消

确定

参数

- event: 是当前设备上报的事件消息。

返回值

- object 类型: 构造一个下发到目标设备的命令消息。
- null: 表示不下发命令到设备。

向其他设备下发属性

该操作用于向其它设备下发属性，实现设备联动功能。

设备来源类型为：设备

操作配置



下发到设备

请选择设备



函数内容

```
1 function main(event) {  
2     /**  
3      * event:      传入事件对象，由硬件上报  
4      * push_attributes: 构造下发的属性对象，作为函数返回值，下发到硬件  
5      */  
6     var push_attributes = {  
7  
8     };  
9     return push_attributes;  
10 }
```

[测试云函数](#) ^

取消

确定

当前设备上报的事件消息，会作为 event 参数传入下发属性的构造函数，您可以编写一定的逻辑，来构造下发到其它设备的属性 attributes。

例如，当前设备上报的事件消息为：

```
{  
  "method": "switchRelay",  
  "param": {  
    "state1": true,  
    "state2": false  
  }  
}
```

该操作的属性下发函数：

```
function main(event) {  
  var push_attributes = {  
    relay1_state: event.params.state1 || false,  
    relay2_state: event.params.state2 || false,  
  };  
  return push_attributes;  
}
```

将事件参数中的字段直接复制到下发的属性消息中，非常简单。更加复杂的场景以此类推即可。

参数

- event: 是当前设备上报的事件消息。

返回值

- object 类型: 构造一个下发到目标设备的命令消息。
- null: 表示不下发命令到设备。

提示

关于如何在设备上接收属性下发，请浏览[云端下发属性至设备](#)。

自定义数据上报

自定义数据上报

云函数内置函数库

云函数内置函数库

EZtCloud 平台在消息规则与任务系统中深度集成了云函数功能，允许开发者通过 JavaScript 脚本实现复杂业务逻辑与数据处理。为进一步提升开发灵活性，平台精心设计了一套功能丰富的内置函数库，涵盖设备属性查询、数据类型转换、时间处理等高频场景，有效降低了云端代码开发的技术门槛。

这些内置工具采用模块化设计，开发者只需在云函数代码中直接调用即可快速实现特定功能。这种开箱即用的特性不仅显著缩短了开发周期，更让代码维护与功能迭代变得轻松高效。通过将通用功能封装为标准接口，平台帮助开发者聚焦核心业务逻辑，在保持系统扩展性的同时提升了整体开发效率。

设备信息

Cloud.getDeviceInfo()

读取当前设备信息

参数

无

返回值

- object类型,设备信息键值对集合，例如：

```
{
  "id": 1017,                // 设备ID
  "category": {...},        // 设备所属设备类型
  "serial_id": null,         // 设备序列号
  "name": "1#照明设备",     // 设备名称
  "username": "DU_d9Si73jjhX", // 设备账号，用户使用设备账号密码进行链接平台设备
  "password": "hWtf1SMKEJ", // 设备密码
  "code": "01",             // 子设备地址，仅对子设备类型有效
  ... ..
}
```

示例

设备属性

Cloud.getCurrentAttributes()

读取设备所有属性当前值。

参数

无

返回值

- object类型,设备信息键值对集合, 例如:

```
{
  "attributes": {                                // 设备属性
    "battery": {"time": 1741662315200,"value": 0},
    "temp": {"time": 1741662315200,"value": true},
    "humi": {"time": 1741662315200,"value": false},
    .....
  },
  "time": 1741662855738                          // 设备属性最后更新时间
}
```

示例

数据类型转换函数

Cloud.int8ToHex(number)

将 8 位**带符号整数**转为十六进制的 HEX 字符串。

参数

- number: 带符号8位整数, 范围从-128~127。

返回值

- string类型: 1个字节的HEX字符串。

示例

```
Cloud.int8ToHex(0)           // 00
Cloud.int8ToHex(127)        // 7F
Cloud.int8ToHex(-128)       // 80
Cloud.int8ToHex(-1)         // FF
```

Cloud.uint8ToHex(number)

将 8 位**无符号整数**转为十六进制的 HEX 字符串。

参数

- number: 无符号8位整数, 范围从0~255。

返回值

- string类型: 1个字节的HEX字符串

示例

```
Cloud.uint8ToHex(0)           // 00
Cloud.uint8ToHex(127)        // 7F
Cloud.uint8ToHex(128)        // 80
Cloud.uint8ToHex(255)        // FF
```

Cloud.int16ToHex(number, endian)

将 16 位**带符号整数**转为十六进制的 HEX 字符串，默认使用**大端字节序**。

参数

- number: 带符号16位整数，范围从-32768~32767。
- endian: 字节序，默认为大端字节序(' > ')，可选参数有大端字节序' > '，小端字节序' < '。

返回值

- string类型: 2个字节的HEX字符串。

示例

```
// 默认使用大端字节序
Cloud.int16ToHex(-32768)      // 8000
Cloud.int16ToHex(-1)         // FFFF
Cloud.int16ToHex(0)          // 0000
Cloud.int16ToHex(1)          // 0001
Cloud.int16ToHex(32767)      // 7FFF
// 使用小端字节序
Cloud.int16ToHex(-32768, '<') // 0080
Cloud.int16ToHex(-1, '<')    // FFFF
Cloud.int16ToHex(0, '<')     // 0000
Cloud.int16ToHex(1, '<')     // 0100
Cloud.int16ToHex(32767, '<') // FF7F
```

Cloud.uint16ToHex(number, endian)

将 16 位**无符号整数**转为十六进制的 HEX 字符串，默认使用**大端字节序**。

参数

- number: 无符号16位整数，范围从0~65535。
- endian: 字节序，默认为大端字节序(' > ')，可选参数有大端字节序' > '，小端字节序' < '。

返回值

- string类型: 2个字节的HEX字符串。

示例

```
// 默认使用大端字节序
Cloud.uint16ToHex(0)           // 0000
Cloud.uint16ToHex(1)           // 0001
Cloud.uint16ToHex(32767)       // 7FFF
Cloud.uint16ToHex(65535)       // FFFF
// 使用小端字节序
Cloud.uint16ToHex(0, '<')       // 0000
Cloud.uint16ToHex(1, '<')       // 0100
Cloud.uint16ToHex(32767, '<')   // FF7F
Cloud.uint16ToHex(65535, '<')   // FFFF
```

Cloud.int32ToHex(number, endian)

将 32 位**带符号整数**转为十六进制的 HEX 字符串，默认使用**大端字节序**。

参数

- number: 带符号32位整数，范围从-2147483648~2147483647。
- endian: 字节序，默认为大端字节序('ABCD')，可选参数有大端字节序' ABCD' ，小端字节序' DCBA' 和两个混合字节序' BADC' (大端模式字节交换)、' CDAB' (小端模式字节交换)。

返回值

- string类型: 4个字节的HEX字符串。

示例

```
// 默认使用大端字节序
Cloud.int32ToHex(0)           // 00000000
Cloud.int32ToHex(1)           // 00000001
Cloud.int32ToHex(2147483647)   // 7FFFFFFF
Cloud.int32ToHex(-2147483648)  // 80000000
Cloud.int32ToHex(-1)           // FFFFFFFF
// 使用小端字节序
Cloud.int32ToHex(0, 'DCBA')    // 00000000
Cloud.int32ToHex(1, 'DCBA')    // 01000000
Cloud.int32ToHex(2147483647, 'DCBA') // FFFFFFFF
Cloud.int32ToHex(-2147483648, 'DCBA') // 00000080
Cloud.int32ToHex(-1, 'DCBA')   // FFFFFFFF
// 使用混合字节序'BADC' (大端模式字节交换)
Cloud.int32ToHex(0, 'BADC')    // 00000000
Cloud.int32ToHex(1, 'BADC')    // 00000100
Cloud.int32ToHex(2147483647, 'BADC') // FF7FFFFFFF
Cloud.int32ToHex(-2147483648, 'BADC') // 00800000
Cloud.int32ToHex(-1, 'BADC')   // FFFFFFFF
// 使用混合字节序'CDAB' (小端模式字节交换)
Cloud.int32ToHex(0, 'CDAB')    // 00000000
Cloud.int32ToHex(1, 'CDAB')    // 00010000
```

```
Cloud.int32ToHex(2147483647, 'CDAB') // FFFF7FFF
Cloud.int32ToHex(-2147483648, 'CDAB') // 00008000
Cloud.int32ToHex(-1, 'CDAB') // FFFFFFFF
```

Cloud.uint32ToHex(number, endian)

将 32 位**无符号整数**转为十六进制的 HEX 字符串，默认使用**大端字节序**。

参数

- number: 无符号32位整数，范围从0~4294967295。
- endian: 字节序，默认为大端字节序(**ABCD**)，可选参数有大端字节序 **ABCD**，小端字节序 **DCBA** 和两个混合字节序 **BADC** (大端模式字节交换)、 **CDAB** (小端模式字节交换)。

返回值

- string类型: 4个字节的HEX字符串。

示例

```
// 默认使用大端字节序
Cloud.uint32ToHex(0) // 00000000
Cloud.uint32ToHex(1) // 00000001
Cloud.uint32ToHex(2147483647) // 7FFFFFFF
Cloud.uint32ToHex(2147483648) // 80000000
Cloud.uint32ToHex(4294967295) // FFFFFFFF
// 使用小端字节序
Cloud.uint32ToHex(0, 'DCBA') // 00000000
Cloud.uint32ToHex(1, 'DCBA') // 01000000
Cloud.uint32ToHex(2147483647, 'DCBA') // FFFFFFFF
Cloud.uint32ToHex(2147483648, 'DCBA') // 00000080
Cloud.uint32ToHex(4294967295, 'DCBA') // FFFFFFFF
// 使用混合字节序'BADC' (大端模式字节交换)
Cloud.uint32ToHex(0, 'BADC') // 00000000
Cloud.uint32ToHex(1, 'BADC') // 00000100
Cloud.uint32ToHex(2147483647, 'BADC') // FF7FFFFFFF
Cloud.uint32ToHex(2147483648, 'BADC') // 00800000
Cloud.uint32ToHex(4294967295, 'BADC') // FFFFFFFF
// 使用混合字节序'CDAB' (小端模式字节交换)
Cloud.uint32ToHex(0, 'CDAB') // 00000000
Cloud.uint32ToHex(1, 'CDAB') // 00010000
Cloud.uint32ToHex(2147483647, 'CDAB') // FFFF7FFF
Cloud.uint32ToHex(2147483648, 'CDAB') // 00008000
Cloud.uint32ToHex(4294967295, 'CDAB') // FFFFFFFF
```

Cloud.float32ToHex(number, endian)

将 32 位 Float 单精度浮点数转为十六进制的 HEX 字符串，默认使用**大端字节序**。

参数

- number: 单精度浮点数。
- endian: 字节序, 默认为大端字节序('**ABCD**'), 可选参数有大端字节序 '**ABCD**' , 小端字节序 '**DCBA**' 和两个混合字节序 '**BADC**' (大端模式字节交换)、 '**CDAB**' (小端模式字节交换)。

返回值

- string类型: 4个字节的HEX字符串。

示例

```
// 默认使用大端字节序
Cloud.float32ToHex(0)           // 00000000
Cloud.float32ToHex(10.123)      // 4121F7CF
Cloud.float32ToHex(-10.123)     // C121F7CF
Cloud.float32ToHex(5000.123)    // 459C40FC
Cloud.float32ToHex(-5000.123)   // C59C40FC
// 使用小端字节序
Cloud.float32ToHex(0, 'DCBA')   // 00000000
Cloud.float32ToHex(10.123, 'DCBA') // CFF72141
Cloud.float32ToHex(-10.123, 'DCBA') // CFF721C1
Cloud.float32ToHex(5000.123, 'DCBA') // FC409C45
Cloud.float32ToHex(-5000.123, 'DCBA') // FC409CC5
// 使用混合字节序'BADC'(大端模式字节交换)
Cloud.float32ToHex(0, 'BADC')   // 00000000
Cloud.float32ToHex(10.123, 'BADC') // 2141CFF7
Cloud.float32ToHex(-10.123, 'BADC') // 21C1CFF7
Cloud.float32ToHex(5000.123, 'BADC') // 9C45FC40
Cloud.float32ToHex(-5000.123, 'BADC') // 9CC5FC40
// 使用混合字节序'CDAB'(小端模式字节交换)
Cloud.float32ToHex(0, 'CDAB')   // 00000000
Cloud.float32ToHex(10.123, 'CDAB') // F7CF4121
Cloud.float32ToHex(-10.123, 'CDAB') // F7CFC121
Cloud.float32ToHex(5000.123, 'CDAB') // 40FC459C
Cloud.float32ToHex(-5000.123, 'CDAB') // 40FCC59C
```

Cloud.double64ToHex(number, endian)

将 64 位 Double 双精度浮点数转为十六进制的 HEX 字符串, 使用**大端字节序**。

参数

- number: 双精度浮点数。
- endian: 字节序, 默认为大端字节序('>'), 可选参数有大端字节序 '>' , 小端字节序 '<' 。

返回值

- string类型: 8个字节的 HEX 字符串。

示例

```
// 默认使用大端字节序
Cloud.Utls.double64ToHex(0) // 0000000000000000
Cloud.Utls.double64ToHex(10000.123) // 40C3880FBE76C8B4
Cloud.Utls.double64ToHex(-10000.123) // C0C3880FBE76C8B4
Cloud.Utls.double64ToHex(500000.123) // 411E84807DF3B646
Cloud.Utls.double64ToHex(-500000.123) // C11E84807DF3B646
// 使用小端字节序
Cloud.Utls.double64ToHexLE(0, '<') // 0000000000000000
Cloud.Utls.double64ToHexLE(10000.123, '<') // B4C876BE0F88C340
Cloud.Utls.double64ToHexLE(-10000.123, '<') // B4C876BE0F88C3C0
Cloud.Utls.double64ToHexLE(500000.123, '<') // 46B6F37D80841E41
Cloud.Utls.double64ToHexLE(-500000.123, '<') // 46B6F37D80841EC1
```

Cloud.hexToInt8(data)

将十六进制的 HEX 字符串转为8 位**带符号整数**。

参数

- data: 1个字节的十六进制 HEX 字符串。

返回值

- number类型: 8 位**带符号整数**。

示例

```
Cloud.hexToInt8('00') // 0
Cloud.hexToInt8('7F') // 127
Cloud.hexToInt8('80') // -128
Cloud.hexToInt8('FF') // -1
```

Cloud.hexToUint8(data)

将十六进制的 HEX 字符串转为8 位**无符号整数**。

参数

- data: 1个字节的 HEX 字符串。

返回值

- number类型: 8 位**无符号整数**。

示例

```
Cloud.hexToUint8('00')    // 00
Cloud.hexToUint8('7F')    // 127
Cloud.hexToUint8('80')    // 128
Cloud.hexToUint8('FF')    // 255
```

Cloud.hexToInt16(data, endian)

将十六进制的 HEX 字符串转为16 位**带符号整数**。

参数

- data: 2 个字节的 HEX 字符串。
- endian: HEX 字符串的字节序，默认使用大端字节序（ ' > ' ）解析，传入（ ' < ' ）使用小端字节序解析。

返回值

- number类型: 16 位**带符号整数**。

示例

```
// 默认使用大端字节序
Cloud.hexToInt16('8000')    // -32768
Cloud.hexToInt16('FFFF')    // -1
Cloud.hexToInt16('0000')    // 0
Cloud.hexToInt16('0001')    // 1
Cloud.hexToInt16('7FFF')    // 32767
// 使用小端字节序
Cloud.hexToInt16('0080', '<') // -32768
Cloud.hexToInt16('FFFF', '<') // -1
Cloud.hexToInt16('0000', '<') // 0
Cloud.hexToInt16('0100', '<') // 1
Cloud.hexToInt16('FF7F', '<') // 32767
```

Cloud.hexToUint16(data, endian)

将十六进制的 HEX 字符串转为16 位**无符号整数**。

参数

- data: 2 个字节的 HEX 字符串。
- endian: HEX 字符串的字节序，默认使用大端字节序（ ' > ' ）解析，传入（ ' < ' ）使用小端字节序解析。

返回值

- number类型: 16 位**无符号整数**。

示例

```
// 默认使用大端字节序
Cloud.hexToUint16('0000')           // 0
Cloud.hexToUint16('0001')           // 1
Cloud.hexToUint16('7FFF')           // 32767
Cloud.hexToUint16('FFFF')           // 65535
// 使用小端字节序
Cloud.hexToUint16('0000', '<')       // 0
Cloud.hexToUint16('0100', '<')       // 1
Cloud.hexToUint16('FF7F', '<')       // 32767
Cloud.hexToUint16('FFFF', '<')       // 65535
```

Cloud.hexToInt32(data, endian)

将十六进制的 HEX 字符串转为32 位**带符号整数**。

参数

- data: 4 个字节的 HEX 字符串。
- endian: HEX 字符串的字节序，默认为大端字节序(‘ABCD’)解析，可选参数有大端字节序‘ABCD’，小端字节序‘DCBA’和两个混合字节序‘BADC’(大端模式字节交换)、‘CDAB’(小端模式字节交换)。

返回值

- number类型: 32 位**带符号整数**。

示例

```
// 默认使用大端字节序
Cloud.hexToInt32('00000000')         // 0
Cloud.hexToInt32('00000001')         // 1
Cloud.hexToInt32('7FFFFFFF')         // 2147483647
Cloud.hexToInt32('80000000')         // -2147483648
Cloud.hexToInt32('FFFFFFFF')         // -1
// 使用小端字节序
Cloud.hexToInt32('00000000', 'DCBA') // 0
Cloud.hexToInt32('01000000', 'DCBA') // 1
Cloud.hexToInt32('FFFFFF7F', 'DCBA') // 2147483647
Cloud.hexToInt32('00000080', 'DCBA') // -2147483648
Cloud.hexToInt32('FFFFFFFF', 'DCBA') // -1
// 使用混合字节序'BADC'(大端模式字节交换)
Cloud.hexToInt32('00000000', 'BADC') // 0
Cloud.hexToInt32('00000100', 'BADC') // 1
Cloud.hexToInt32('FF7FFFFFFF', 'BADC') // 2147483647
Cloud.hexToInt32('00800000', 'BADC') // -2147483648
Cloud.hexToInt32('FFFFFFFF', 'BADC') // -1
// 使用混合字节序'CDAB'(小端模式字节交换)
Cloud.hexToInt32('00000000', 'CDAB') // 0
Cloud.hexToInt32('00010000', 'CDAB') // 1
```

```
Cloud.hexToInt32('FFFF7FFF', 'CDAB') // 2147483647
Cloud.hexToInt32('00008000', 'CDAB') // -2147483648
Cloud.hexToInt32('FFFFFFFF', 'CDAB') // -1
```

Cloud.hexToUint32(data, endian)

将十六进制的 HEX 字符串转为32 位**无符号整数**。

参数

- data: 4 个字节的 HEX 字符串。
- endian: HEX 字符串的字节序，默认为大端字节序（'ABCD'）解析，可选参数有大端字节序'ABCD'，小端字节序'DCBA'和两个混合字节序'BADC'（大端模式字节交换）、'CDAB'（小端模式字节交换）。

返回值

- number类型: 32 位**无符号整数**。

示例

```
// 默认使用大端字节序
Cloud.hexToUint32('00000000') // 0
Cloud.hexToUint32('00000001') // 1
Cloud.hexToUint32('7FFFFFFF') // 2147483647
Cloud.hexToUint32('80000000') // 2147483648
Cloud.hexToUint32('FFFFFFFF') // 4294967295
// 使用小端字节序
Cloud.hexToUint32('00000000', 'DCBA') // 0
Cloud.hexToUint32('01000000', 'DCBA') // 1
Cloud.hexToUint32('FFFFFFFF7F', 'DCBA') // 2147483647
Cloud.hexToUint32('00000080', 'DCBA') // 2147483648
Cloud.hexToUint32('FFFFFFFF', 'DCBA') // 4294967295
// 使用混合字节序'BADC'（大端模式字节交换）
Cloud.hexToUint32('00000000', 'BADC') // 0
Cloud.hexToUint32('00000100', 'BADC') // 1
Cloud.hexToUint32('FF7FFFFFFF', 'BADC') // 2147483647
Cloud.hexToUint32('00800000', 'BADC') // 2147483648
Cloud.hexToUint32('FFFFFFFF', 'BADC') // 4294967295
// 使用混合字节序'CDAB'（小端模式字节交换）
Cloud.hexToUint32('00000000', 'CDAB') // 0
Cloud.hexToUint32('00010000', 'CDAB') // 1
Cloud.hexToUint32('FFFF7FFF', 'CDAB') // 2147483647
Cloud.hexToUint32('00008000', 'CDAB') // 2147483648
Cloud.hexToUint32('FFFFFFFF', 'CDAB') // 4294967295
```

Cloud.hexToFloat32(data, endian)

将十六进制的 HEX 字符串转为32 位**单精度浮点数**。

参数

- data: 4 个字节的 HEX 字符串。
- endian: HEX 字符串的字节序, 默认为大端字节序('ABCD')解析, 可选参数有大端字节序 'ABCD' , 小端字节序 'DCBA' 和两个混合字节序 'BADC' (大端模式字节交换)、 'CDAB' (小端模式字节交换)。

返回值

- number类型: 32 位**单精度浮点数**。

示例

```
// 默认使用大端字节序
Cloud.hexToFloat32('00000000')           // 0
Cloud.hexToFloat32('4121F7CF')           // 10.123
Cloud.hexToFloat32('C121F7CF')           // -10.123
Cloud.hexToFloat32('459C40FC')           // 5000.123
Cloud.hexToFloat32('C59C40FC')           // -5000.123
// 使用小端字节序
Cloud.hexToFloat32('00000000', 'DCBA')    // 0
Cloud.hexToFloat32('CFF72141', 'DCBA')    // 10.123
Cloud.hexToFloat32('CFF721C1', 'DCBA')    // -10.123
Cloud.hexToFloat32('FC409C45', 'DCBA')    // 5000.123
Cloud.hexToFloat32('FC409CC5', 'DCBA')    // -5000.123
// 使用混合字节序'BADC'(大端模式字节交换)
Cloud.hexToFloat32('00000000', 'BADC')    // 0
Cloud.hexToFloat32('2141CFF7', 'BADC')    // 10.123
Cloud.hexToFloat32('21C1CFF7', 'BADC')    // -10.123
Cloud.hexToFloat32('9C45FC40', 'BADC')    // 5000.123
Cloud.hexToFloat32('9CC5FC40', 'BADC')    // -5000.123
// 使用混合字节序'CDAB'(小端模式字节交换)
Cloud.hexToFloat32('00000000', 'CDAB')    // 0
Cloud.hexToFloat32('F7CF4121', 'CDAB')    // 10.123
Cloud.hexToFloat32('F7CFC121', 'CDAB')    // -10.123
Cloud.hexToFloat32('40FC459C', 'CDAB')    // 5000.123
Cloud.hexToFloat32('40FCC59C', 'CDAB')    // -5000.123
```

Cloud.hexToDouble64(data, endian)

将 十六进制的 HEX 字符串转为64 位**双精度浮点数**。

参数

- data: 8个字节的 HEX 字符串。
- endian: 字节序, 默认为大端字节序('>'), 可选参数有大端字节序 '>' , 小端字节序 '<' 。

返回值

- number类型: 64位**双精度浮点数**。

示例

```
// 默认使用大端字节序
Cloud.Utls.double64ToHex('0000000000000000') // 0
Cloud.Utls.double64ToHex('40C3880FBE76C8B4') // 10000.123
Cloud.Utls.double64ToHex('C0C3880FBE76C8B4') // -10000.123
Cloud.Utls.double64ToHex('411E84807DF3B646') // 500000.123
Cloud.Utls.double64ToHex('C11E84807DF3B646') // -500000.123
// 使用小端字节序
Cloud.Utls.double64ToHex('0000000000000000', '<') // 0
Cloud.Utls.double64ToHex('B4C876BE0F88C340', '<') // 10000.123
Cloud.Utls.double64ToHex('B4C876BE0F88C3C0', '<') // -10000.123
Cloud.Utls.double64ToHex('46B6F37D80841E41', '<') // 500000.123
Cloud.Utls.double64ToHex('46B6F37D80841EC1', '<') // -500000.123
```

Cloud.hexToJson(data)

将十六进制的 HEX 字符串转为 **JSON对象**。

参数

- data: string类型 有效的 HEX 字符串。

返回值

- object类型: JSON对象。

示例

```
Cloud.hexToJson('7b226e616d65223a22e6b8a9e6b9bfe5baa6e79b91e6b58be4bca0e6849fe599a8222c2274656d7065726174757265223a33342c2268756d6964697479223a36347d')

// {"name":"温湿度监测传感器","temperature":34,"humidity":64}
```

Cloud.hexToStr(data)

将十六进制的 HEX 转为 字符串。

参数

- data: string类型 有效的 HEX 字符串。

返回值

- string类型。

示例

```
Cloud.hexToStr("6162636431323334") // abcd1234
```

Cloud.hexToBuffer(data)

将十六进制 HEX 字符串转为 Buffer 字节序列。

参数

- data: string类型 有效的 HEX 字符串。

返回值

- Buffer 字节流类型。

示例

```
Cloud.hexToBuffer("4150494F3D3432") // Buffer 字节流 [65,80,73,79,61,52,50]
```

Cloud.bufferToHex(data)

将 Buffer 字节序列转为十六进制 HEX 字符串。

参数

- data: Buffer 字节流类型。

返回值

- string 类型: 表示十六进制 HEX。

示例

```
// data 为 Buffer 字节流  
Cloud.bufferToHex(data) // 4150494F3D3432
```

Cloud.binToInt(data)

将 二进制序列转为十进制无符号整数。

参数

- data: 包含二进制数值的数组, 例如: [0,1,0,0,1,0,0,0]。

返回值

- number 类型: 无符号整数

示例

```
Cloud.binToInt([0,0,0,0,0,0,0,1])    // 1
Cloud.binToInt([0,0,0,0,0,0,1,1])    // 3
Cloud.binToInt([0,0,0,0,1,1,1,1])    // 15
```

Cloud.binToHex(data)

将二进制序列转为十六进制 HEX 字符串。

参数

- data: 包含二进制数值的数组, 例如: [0,1,0,0,1,0,0,0]。

返回值

- string 类型: 十六进制 HEX 字符串。

示例

```
Cloud.binToHex([0,0,0,0,0,0,0,1])    // 01
Cloud.binToHex([0,0,0,0,0,0,1,1])    // 03
Cloud.binToHex([0,0,0,0,1,1,1,1])    // 0F
```

Cloud.hexToBin(data)

将十六进制 HEX 字符串转为二进制序列。

参数

- string 类型: 十六进制 HEX 字符串。

返回值

- object 类型: 包含二进制数值的数组, 例如: [0,1,0,0,1,0,0,0]。

示例

```
Cloud.binToHex('01')    // [0,0,0,0,0,0,0,1]
Cloud.binToHex('03')    // [0,0,0,0,0,0,1,1]
Cloud.binToHex('0F')    // [0,0,0,0,1,1,1,1]
```

Cloud.hexToBase64(data)

将 HEX 字符串转为 Base64 字符串。

参数

- data: 有效的十六进制 HEX 字符串。

返回值

- string 类型: Base64 字符串。

示例

```
Cloud.hexToBase64("4150494F3D3432")    // QVBJTz00Mg==
```

Cloud.bufferToBase64(data)

将 Buffer 字节流类型转为 Base64 字符串。

参数

- data: 有效的 Base64 字符串。

返回值

- string 类型: Base64 字符串。

示例

```
// data 二进制数据流
Cloud.bufferToBase64(data)    // QVBJTz00Mg==
```

Cloud.strToBase64(data)

将字符串转为 base64 编码的字符串。

参数

- data: string 字符串类型。

返回值

- string 类型, 表示 base64 编码的字符串。

示例

```
Cloud.strToBase64("temperature=23.1,humitidy=56.2")
// dGVtcGVyYXR1cmU9MjMuMSxodW1pdG1keT01Ni4y
```

Cloud.base64ToStr(data)

将 Base64 字符串解码为字符串。

参数

- data: 有效的 Base64 字符串。

返回值

- string 类型。

示例

```
Cloud.base64ToHex("dGVtcGVyYXR1cmU9MjMuMSxodW1pdGlkeT01Ni4y")  
// temperature=23.1,humitidy=56.2
```

Cloud.base64ToJson(data)

将 Base64 字符串解码为JSON对象。

参数

- data: 有效的 Base64 字符串。

返回值

- object 类型: JSON对象。

示例

```
Cloud.base64ToJson('eyJ1YW1lIjogIkpvaG4iLCAiYWdlIjogMzAsICJjaXR5IjogIk5ldyBZb3JrIn0=')  
// {"age": 30,"city": "New York","name": "John"}
```

Cloud.base64ToHex(data)

将 Base64 字符串转为 HEX 十六进制字符串。

参数

- data: 有效的 Base64 字符串。

返回值

- string 类型: HEX 字符串。

示例

```
Cloud.base64ToHex("QVBJTz00Mg==") // 4150494f3d3432
```

Cloud.base64ToBuffer(data)

将 Base64 字符串转为 Buffer 字节序列。

参数

- data: 有效的 Base64 字符串。

返回值

- Buffer 字节流类型。

示例

```
Cloud.base64ToBuffer("QVBJTz00Mg==") // Buffer 字节流 [65,80,73,79,61,52,50]
```

Modbus RTU函数

Cloud.CRC16Modbus(data)

生成Modbus RTU报文的CRC16校验码。

参数

- data: string类型 Modbus RTU报文, 例如: '01 03 00 02 00 02'。

返回值

- string类型: CRC16校验码, 例如: 65 CB。

示例

```
Cloud.CRC16Modbus('01 03 00 02 00 02')  
// 65 CB
```

创建任务

创建任务

属性下发任务

属性下发任务

命令下发任务

命令下发任务

自定义下发任务

自定义下发任务

告警规则

告警规则

EZtCloud提供了内置的告警规则，可以将设备上报属性的变化作为告警触发条件。可以为接入的设备创建灵活的告警规则，从而实现告警通知。

注：除此之外，依托于开放的 EZtCloud API，也可以应用端实现任意告警逻辑和通知。

创建规则

可以在**设备详情页>告警>告警规则**中，为当前设备创建告警规则。例如：



也可以在**设备类型详情页>告警**中，为当前设备类型创建告警规则。

以上两种方式的区别是设备源的不同，设备源决定了告警规则的作用范围，也就是对哪些设备有效。

告警规则的设备源

告警规则支持两种设备源类型：

- 设备类型：告警规则的作用范围包括设备类型下的所有设备。例如：设备源选择燃气浓度传感器设备类型，那么该类型下的所有燃气浓度传感器设备，都可使用该告警规则，而无需为每个设备重复创建告警规则。
- 设备：告警规则的作用范围仅针对指定的一个或多个设备。

告警触发条件

触发条件是告警规则中的重要部分，这里支持设置多个属性触发条件。如下图：

编辑告警规则

×

逻辑条件

且

或

告警触发条件

温度

大于

30

删除

请选择属性

请选择条件

字符串

删除

添加条件

重复次数

无

持续时间

无

设置多个触发条件

当设置多个属性触发条件时，请注意逻辑关系，您可以设置 **且** 或者 **或**。

- 且：表示多个条件要同时成立才会触发，例如：当湿度大于50% 并且温度大于 20℃时触发告警。
- 或：表示多个条件只要有一个成立即可触发，例如：当湿度大于50%或温度大于 20℃时触发告警。

重复次数和持续时间

在满足触发条件的同时，还支持设置重复次数和持续时间。

- 重复次数：设置重复次数后，告警条件首次触发时不会进入告警状态，也不会发送告警通知。当告警条件连续重复触发该次数后，进入告警状态并发送告警通知。
- 持续时间：设置持续时间后，系统会在告警条件被连续触发达到该持续时间后，进入告警状态。

且

或

告警触发条件

温度

大于

30

删除

添加条件

重复次数

无

▼

持续时间

无

⊗

* 告警级别

普通告警

重要告警

紧急告警

⚠ 注意

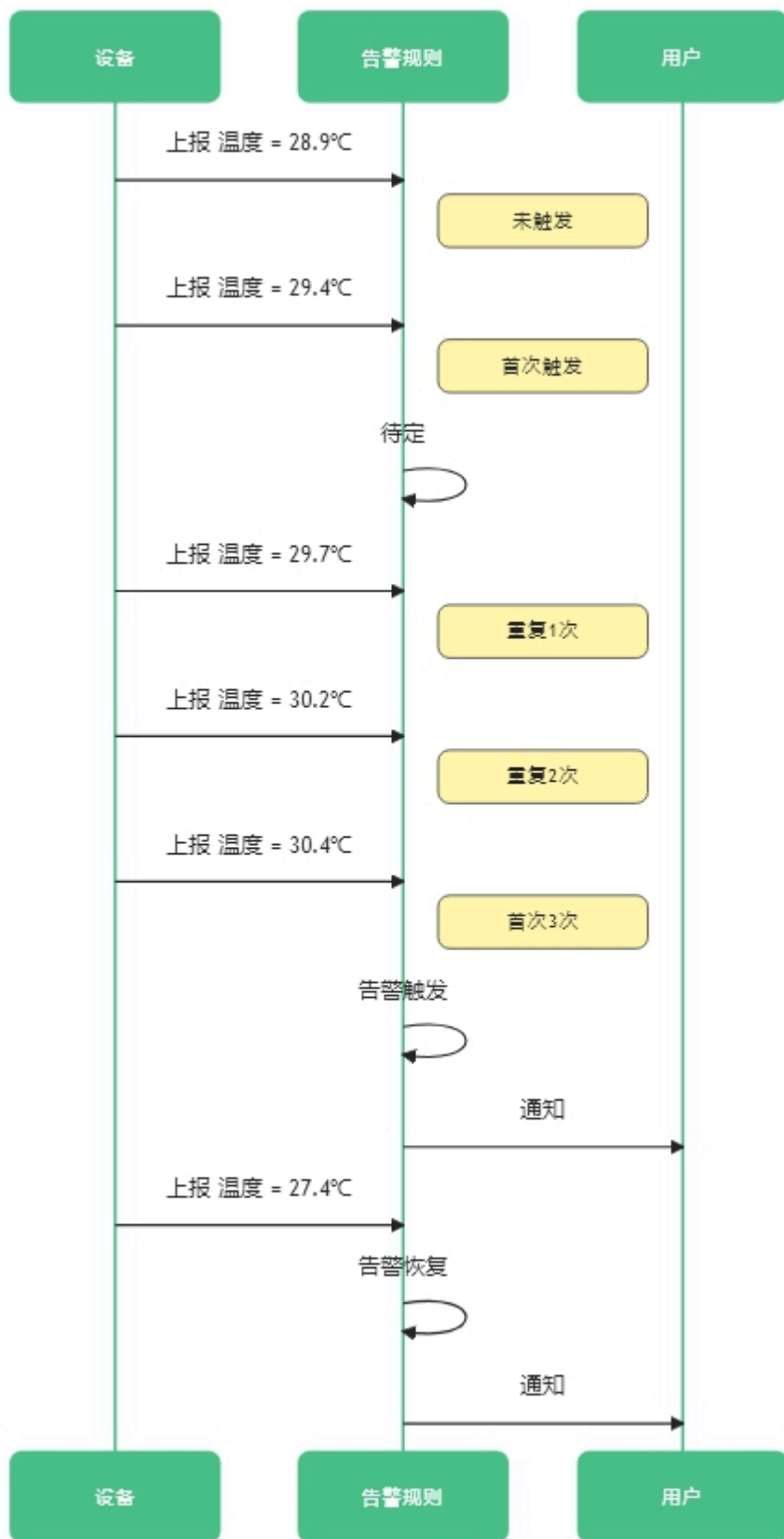
当重复次数和持续时间同时设置时，两者必须同时满足，设备才会进入告警状态。

举个例子：告警规则设置如下：

触发条件是温度大于 29℃

重复次数是 3 次

时序图如下：



告警级别

告警级别用来区分告警的重要级别，用在告警历史和告警通知的显示文字中。例如，在邮件通知方式中，告警级别会显示在邮件标题中。

可选的告警级别包括：

- 普通告警
- 重要告警
- 紧急告警

通知组

将已创建的通知组加入告警规则中，用于发送告警通知。支持设置多个通知组。为了防止频繁通知造成不必要消耗，平台限值了对每日通知次数上限(默认最大值1000)和单个设备每日通知次数上限(默认最大值100)

关于通知组的详细介绍，请浏览[告警通知](#)



The image shows a configuration dialog box titled "通知组" (Notification Group). It contains a dropdown menu at the top with a placeholder icon and the text "通知". Below the dropdown, there are two settings, each with a red asterisk indicating they are required:

- * 每日通知次数上限** (Maximum daily notification count): A numeric input field showing "1000" with up and down arrow buttons.
- * 单个设备每日通知次数上限** (Maximum daily notification count per device): A numeric input field showing "100" with up and down arrow buttons.

At the bottom of the dialog, there are two buttons: a blue "更新" (Update) button and a white "取消" (Cancel) button.

告警规则的可用状态

- 全局可用状态：每个告警规则可设置全局可用状态，用来启用或禁用该规则，对该告警规则的所有设备源都生效。
- 单个设备可用状态：在全局可用状态开启的情况下，您可以对关联的设备独立设置启用或禁用状态。例如，当对某个设备进行维护时，可临时关闭该设备的告警规则，但不影响告警规则关联的其它设备。

告警规则的告警状态

告警规则拥有以下几种的告警状态：

- 告警 (Alerting)：表示最近一次设备属性上报已触发告警规则，且达到设置的重复次数和持续时间。如果未设置重复次数和持续时间，则首次触发会进入告警状态。
- 待定 (Pending)：表示最近一次设备属性上报已触发告警规则，但未达到设置的重复次数和持续时间。
- 恢复 (Restore)：表示曾经触发了告警和待定，最近一次设备属性上报未触发告警规则。

告警通知

告警通知

EZtCloud提供了内置的告警通知方式，包括：

- 邮件
- 短信
- 电话
- 微信公众号
- 钉钉群机器人

要使用这些通知方式，我们需要创建通知组。

创建通知组

进入 **告警设置 > 通知组 > 创建通知组**，即可快速创建通知组。

在创建过程中，选择您需要的通知方式，一个通知组可以选择多个通知方式。

选择通知方式

项目成员通知(项目设置>项目成员)

可以快速对项目成员开启以下通知方式：

- 邮箱通知：使用EZtCloud注册邮箱。
- 微信公众号通知：面向已完成微信公众号绑定的项目成员。
- 短信通知：面向完成手机号认证的项目成员。
- 钉钉工作通知：面向加入钉钉虚拟组织的项目成员。

项目成员通知

账号	昵称	邮件通知	短信通知	微信通知	钉钉工作通知
lianghui	梁辉	☐	☐	☐	☐
songlangshan	宋立强	☐	☒	☒	☒
zhewenqiang	周文强	☐	☐	☐	☐
wei1	魏东	☐	☒	☒	☒
wang	王和群	☐	☐	☐	☐

钉钉机器人通知

需要填写钉钉群机器人Webhook地址和钉钉机器人密钥

* 钉钉机器人通知



设置钉钉机器人URL

https://

设置钉钉机器人密钥

密钥key

获取钉钉群Webhook地址和钉钉机器人密钥步骤如下：

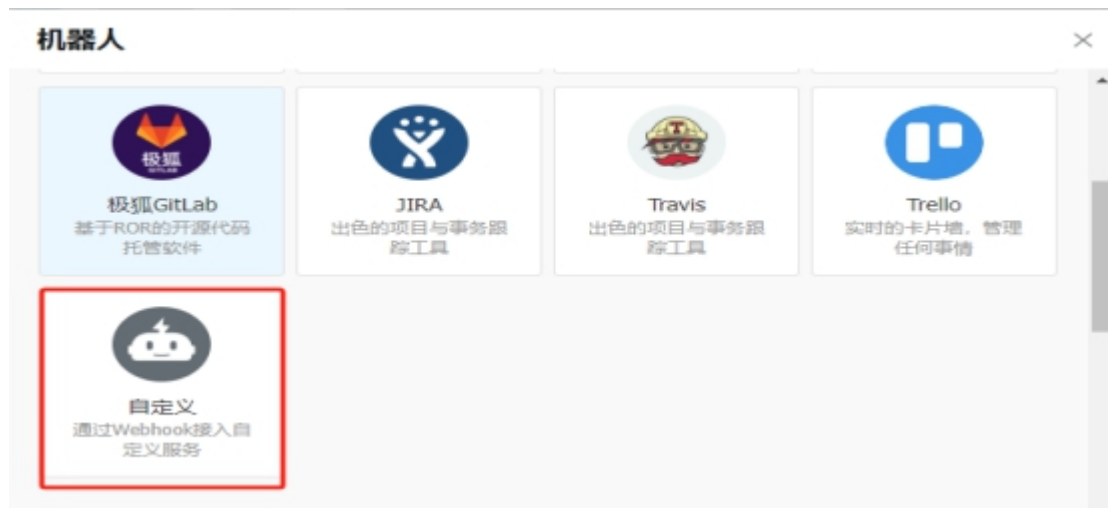
群设置>添加机器人>自定义>添加>加签>完成



添加机器人：



自定义：



加签:



完成:



1.添加机器人✓

2.设置webhook，点击设置说明查看如何配置以使机器人生效

Webhook:

https://oapi.dingtalk.com/robot/send?access t

复制

复制备用

* 请保管好此 Webhook 地址，不要公布在外部网站上，泄露有安全风险
使用 Webhook 地址，向钉钉群推送消息

完成

设置说明

告警历史

告警历史

设备一旦触发告警规则，云平台便会记录告警消息。

可以在告警历史中浏览所有告警消息，也可以通过指定**告警级别/告警状态/告警设备**来搜索告警历史。

告警记录

不限告警状态

不限告警级别

不限设备

回

开始日期

至

结束日期

查询

告警状态	告警级别	告警规则	告警设备	告警内容	告警描述	执行时间
恢复	故障	温度高	温度传感器	温度:10℃		2024-03-01 17:18:11.047
告警	故障	温度高	温度传感器	温度:50℃		2024-03-01 17:17:11.730
恢复	故障	温度高	温度传感器	温度:12℃		2024-03-01 17:17:00.232
告警	故障	温度高	温度传感器	温度:40℃		2024-03-01 14:52:52.179
恢复	故障	温度高	温度传感器	温度:10℃		2024-03-01 14:02:09.659
告警	故障	温度高	温度传感器	温度:40℃		2024-02-29 21:29:44.677
恢复	故障	温度过高告警	温度传感器1	温度:3℃		2024-02-24 10:11:13.115
告警	故障	温度过高告警	温度传感器1	温度:30℃		2024-02-24 10:07:26.052
恢复	故障	温度过高告警	温度传感器1	温度:10℃		2024-02-24 10:07:01.280
告警	故障	温度过高告警	温度传感器1	温度:30℃		2024-02-24 10:06:16.125

1

 页

创建规则

在项目资源和设备概要中，可以快速浏览告警历史统计，帮助您直观的分析在过去的不同时段中，设备告警的出现频次。



OTA固件升级

OTA固件升级

什么是OTA?

OTA 的全称是 Over-the-Air Technology，即空中升级技术，也就是我们通常说的远程升级，在物联网硬件中特指对固件或系统的升级，有时也称为 FOTA，即 Firmware OTA。

EZtCloud OTA 有哪些特性?

- 版本号支持格式多样化，云平台提供版本号检查，设备端无需任何计算。
- 支持设备轮询升级检查（Poll）和云端推送升级（Push）两种模式。
- 支持云端批量升级同一设备类型下的多台设备。
- 支持云端定时推送升级任务到指定设备或所有设备。
- 可指定一个或多个待升级设备，适用于灰度升级和局部调试。
- 可指定待升级版本和目标版本，适用于差分升级。
- 提供升级包 md5 等校验计算。
- 固件下载支持 http/https 可选。

OTA 固件版本管理

EZtCloud 提供了独立的固件托管服务，您只需创建固件版本，并上传固件包。

创建固件版本

进入 [维护](#) > [OTA 版本](#) > [创建 OTA 版本](#)，填写如下内容：

创建OTA版本



* 版本名称

master-v1.2.1

* 设备类型

LoRaWAN网关(UG系列)-245

* 版本号

1.2.1

* 模块

main

* 是否指定待升级设备

是否

* 是否指定待升级版本

是否

* 固件 ?

上传文件

* 固件下载地址使用HTTPS

是否

* 是否启用

是

否

- 版本名称：给固件版本起一个任意的名称。
- 设备类型：选择需要升级的设备所属设备类型。固件版本对设备类型下的所有设备有效。
- 版本号：支持通用的版本号命名方式，例如：1.0.0
- 模块：用于表示该固件属于哪个模块，默认为 main，即硬件的主模块。当硬件中有多个需要升级的模块时，可使用不同的模块名称来创建固件。

- 是否指定待升级设备：如果不指定待升级设备，则默认该固件版本对设备类型下所有设备有效。

使用该功能，可对新固件限定小范围升级，从而验证升级可靠性。另外，也可以根据也许的需要，实现 灰度升级。

- 是否指定待升级版本：如果不指定待升级版本，则默认该固件版本对设备类型下所有版本的设备有效。

当进行 差分升级 时，需要特别约定固件升级前后的版本，适合使用该功能。

- 上传固件：支持文件格式包括：.bin, .img, .dav, .tar, .gz, .zip, .gzip, .apk, .tar.gz。
- 固件下载地址使用 HTTPS：这里的设置决定了云平台下发给设备的升级命令消息中，固件下载地址是否使用 HTTPS。对于不支持 HTTPS 访问的单片机，可以关闭该选项，使用普通的 HTTP 下载地址。

编辑固件版本

固件版本创建后，支持以下编辑操作：

- 启用/禁用：固件版本被禁用后，则不会被云平台用在新版本检查中，可用于版本临时测试或回滚等场景。
- 编辑指定升级的设备
- 编辑指定升级的版本
- 编辑是否支持 HTTPS
- 移除固件版本

💡 提示

固件版本创建后，不支持重新上传固件包，以确保版本和固件包的一致性，避免引起不必要的混淆。如需上传新的固件包，请创建新的固件版本。

设备上报固件当前版本号

在设备进行 OTA 升级之前，云平台需要知道设备当前固件的版本号，从而为每个设备计算是否需要升级，以及需要升级到哪个新版本。

设备上报固件当前版本号，需要通过 MQTT 接入云平台，通过简单的属性上报，即可上报当前版本号。

下边是一个属性上报的例子，代表版本的属性标识符为 version，实际上您可以使用任意标识符，比如：mcu_version、gps_version。

```
{
  "version": "1.0.0"
}
```

通常，设备应该在每次上电开机，并成功连接云平台后，首先上报当前版本。

OTA升级方式

EZtCloud 云平台支持两种OTA升级方式：

- 设备端定期检查新版本
- 云平台推送新版本

设备端定期检查新版本

由设备端定期向云平台发起检查新版本的请求，云平台回应设备是否存在新版本需要升级。序列图如下：

```
sequenceDiagram
    participant Device
    participant EZtCloud
    participant User

    Device->>Device: 开机
    Note over Device: 上报当前版本
    Device->>EZtCloud: 属性上报 version="1.0.0"
    Note over Device: 定期检查新版本
    loop Every minute
        Device->>EZtCloud: 事件上报 otaCheck
        EZtCloud->>Device: 命令下发 otaUpgrade, upgrade=false
    end
    User->>EZtCloud: 创建固件版本 1.0.1
    Device->>EZtCloud: 事件上报 otaCheck
    Note over Device: 发现新版本
    EZtCloud->>Device: 命令下发 otaUpgrade, upgrade=true, version="1.0.1"
    Device->>Device: 下载固件
    Device->>Device: 更新固件
    Device->>Device: 重启
```

创建OTA事件规则

设备向云平台请求新版本的过程，是借助事件上报消息来实现的。

我们需要创建一个事件上报的规则，来实现 OTA 升级检查。

进入 **规则 > 创建规则**，填写如下内容：

- 规则名称：起一个规则名称，例如：OTA 升级检查。
- 规则类型：选择 **事件上报**。
- 设备来源类型：选择 **设备类型**
- 选择来源：选择您需要 OTA 升级的设备所属的设备类型。
- 事件名称：输入 otaCheck，也可以使用其它您喜欢的标识符，只要和随后事件消息中的标识符一致即可。

如图：

创建规则



* 规则名称

OTA 升级检查

* 规则类型 ?

事件上报

* 设备来源类型

设备类型

设备

* 选择来源(设备类型) ?

LoRaWAN网关(UG系列)

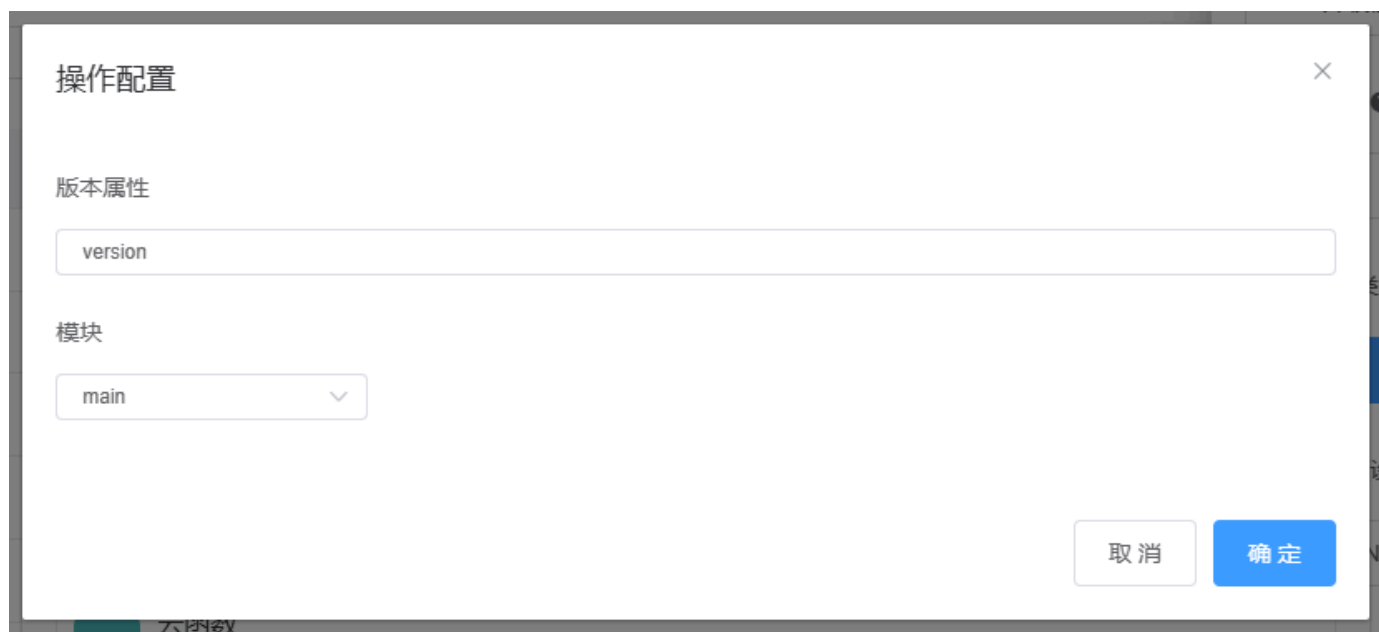
事件标识符

otaCheck

随后给事件规则添加操作，选择**OTA升级检查**，如下图：



在弹出的设置窗口中，填写如下：



- 版本属性：填写属性上报时，代表版本的属性标识符，这里是 version。
- 模块：填写创建固件版本时对应的模块名称，默认是 main。

事件规则创建完成。

设备上报事件消息

接下来，就可以在设备端通过 MQTT 协议向云平台发送事件上报消息，如下：

```
{
  "method": "otaCheck",
  "params": {}
}
```

设备接收命令消息

云平台收到设备上报的事件消息后，会立即在 OTA 固件版本库中查找是否有符合的新版本，如果存在新版本，则下发包含新版本信息的命令消息，例如：

```
{
  "method": "otaUpgrade",
  "params": {
    "module": "main",
    "upgrade": true,
    "version": "1.1.5",
    "url": "http://xxx.com/xxx/xxx.bin",
    "size": 1363713,
    "md5": "481701fa4093499eacd05b57bebc7ffc"
  },
  "command_name": "OTA升级",
  "id": 98310
}
```

云平台推送新版本

另一种 OTA 升级方式是由云平台主动推送包含新版本固件信息的信息给设备端，设备端收到消息后下载固件并完成升级。

这种方式和前一种方式的区别，主要在于不需要设备端主动上报事件来检查新版本，而是由工作人员手动触发或应用端调用 API 来推送包含新版本信息的命令消息给设备端。

序列图如下：

```
sequenceDiagram
    participant Device
    participant EZtCloud
    participant User

    Device->>Device: 开机
    Note over Device: 上报当前版本
    Device->>EZtCloud: 属性上报 version="1.0.0"
    Note over Device: 等待云端下发升级命令
    User->>EZtCloud: 创建固件版本 1.0.1
    User->>EZtCloud: 运行升级任务
    EZtCloud->>Device: 命令下发 otaUpgrade, upgrade=true, version="1.0.1"
    Device->>Device: 下载固件
    Device->>Device: 更新固件
    Device->>Device: 重启
```

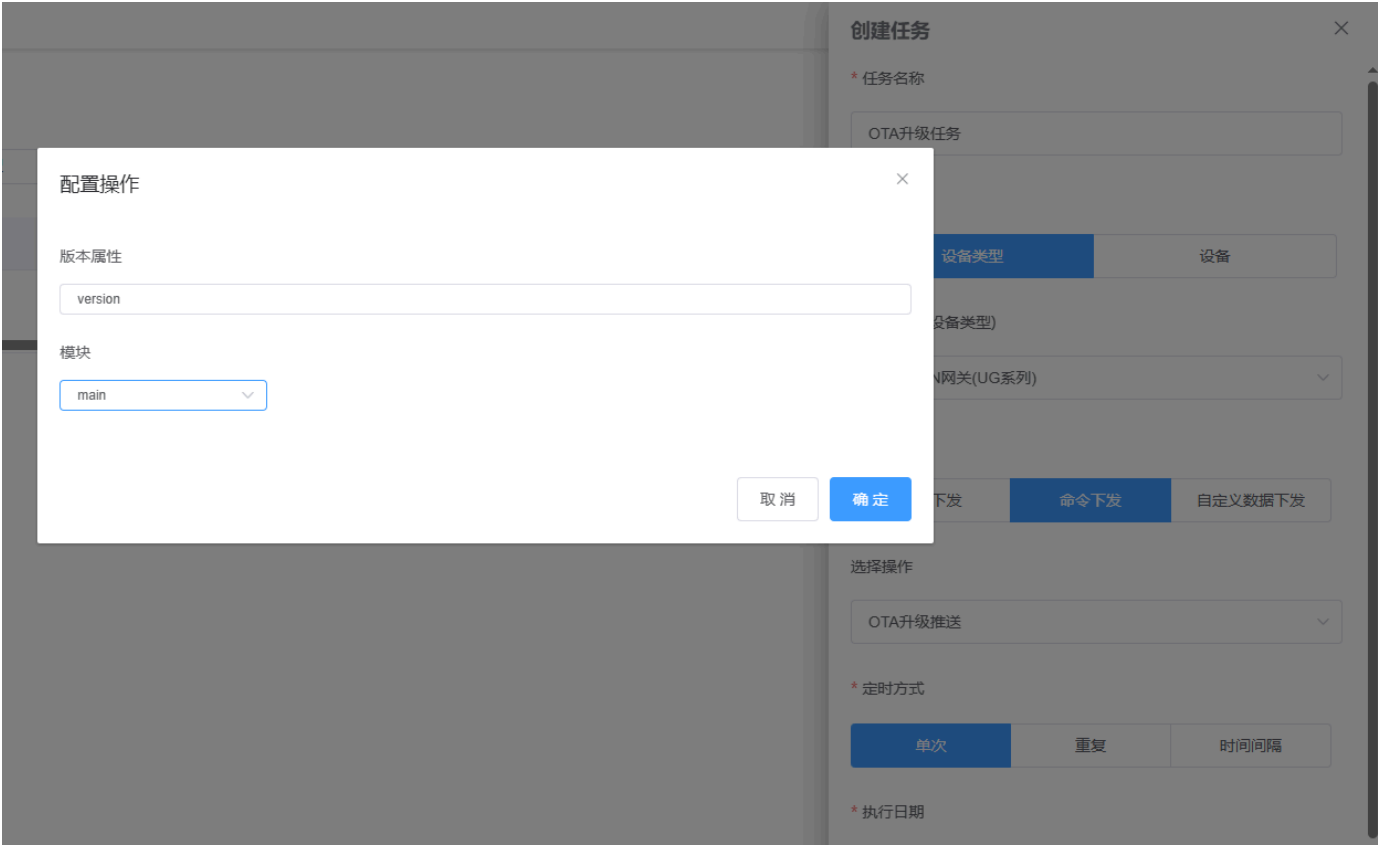
通过云平台主动推送新版本给设备，是通过 **任务** 来实现的。云平台提供了 OTA 升级推送的任务类型，该任务会对运行的目标设备进行版本对比检查，如果版本库中存在新版本可升级，则下发命令消息给设备。

 提示

当设备不存在可升级的新版本时，OTA 升级推送任务不会下发任何命令给设备。

创建OTA升级推送任务

进入 **任务 > 创建任务**，填写如下内容：



- 任务名称：起一个任务名称，例如：OTA 升级任务
- 目标类型：选择设备类型
- 选择目标：选择需要 OTA 升级的设备所属的设备类型。
- 任务类型：选择命令下发
- 选择任务：选择 **OTA 升级推送**
- 定时方式：可选择单次定时，也可根据需要选择相应的定时策略。

在 **OTA 升级推送** 的配置窗口中，类似事件规则的填写：

- 版本属性：填写属性上报时，代表版本的属性标识符，这里是 version。
- 模块：填写创建固件版本时对应的模块名称，默认是 main。

任务创建完成。

运行OTA升级推送任务

进入项目的 **任务** 或当前设备类型的 **任务**，可以看到刚刚创建的任务，点击运行图标，可手动触发运行任务。

运行任务

×

任务名称

OTA升级任务

任务类型

命令下发 - 设备类型

* 目标类型

所有目标设备

指定目标设备

取消

确定

这里可以选择对设备类型下所有设备运行升级推送任务，或者指定一个或多个设备运行该任务。

设备端接收命令消息

这里和设备主动请求新版本时的下发命令完全相同，可参考：[设备接收命令消息](#)。

网关子设备的固件升级

OTA 固件升级功能不仅可用于直连设备（也包括网关本身），还可以用于网关子设备。只需为子设备建立前边提到的相关规则和任务，并为子设备的设备类型创建固件版本信息。

例如，我们为子设备类型创建了事件规则，绑定在子设备的 otaCheck 事件。如下图：

← 返回 | 类型详情

子设备类型

ID: 246 设备接入类型: 子设备

关联设备

功能定义

自定义数据流

规则

任务

告警

个性化UI配置

设置

不限规则类型

不限规则状态

创建规则

刷新

规则名称	规则类型	设备来源类型	规则操作	启用规则	创建时间	修改时间
子设备固件升级	事件上报	设备类型			2024-08-08 10:33:41	2024-08-08 10:33:41

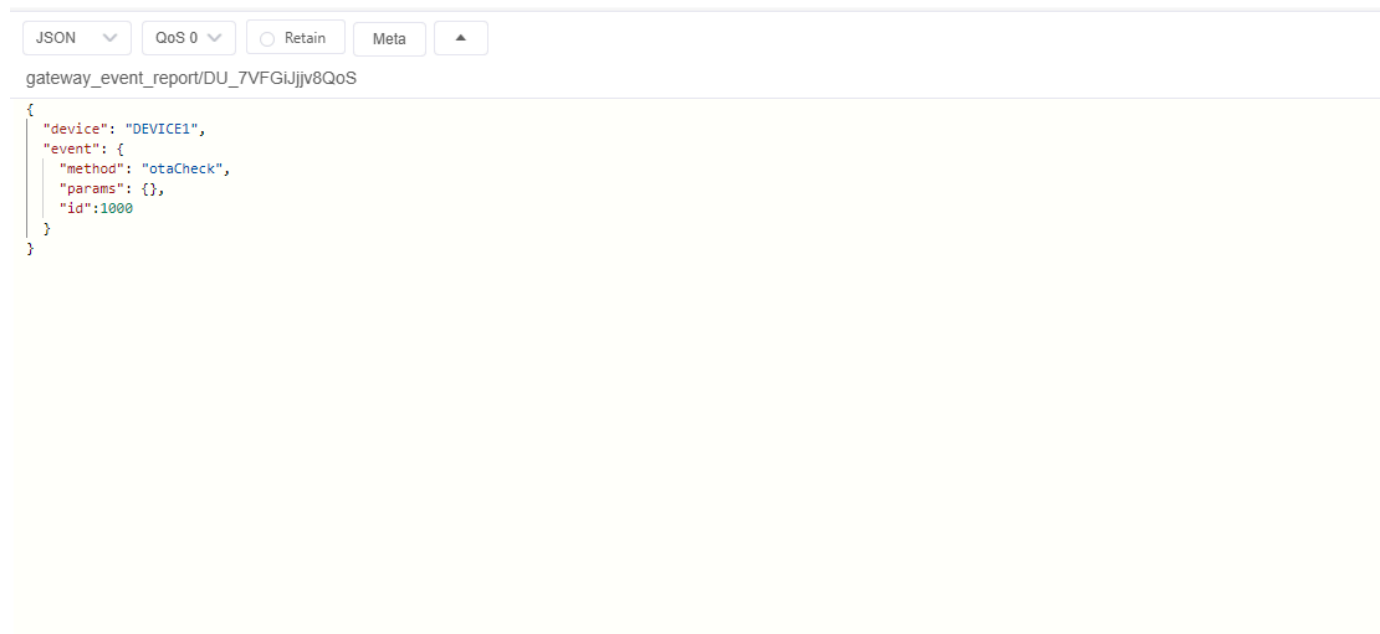
 提示

在设备上报事件检查固件新版本之前，别忘了先通过属性上报，来上报子设备的当前版本号。详细介绍请浏览[设备上报固件当前版本号](#)。

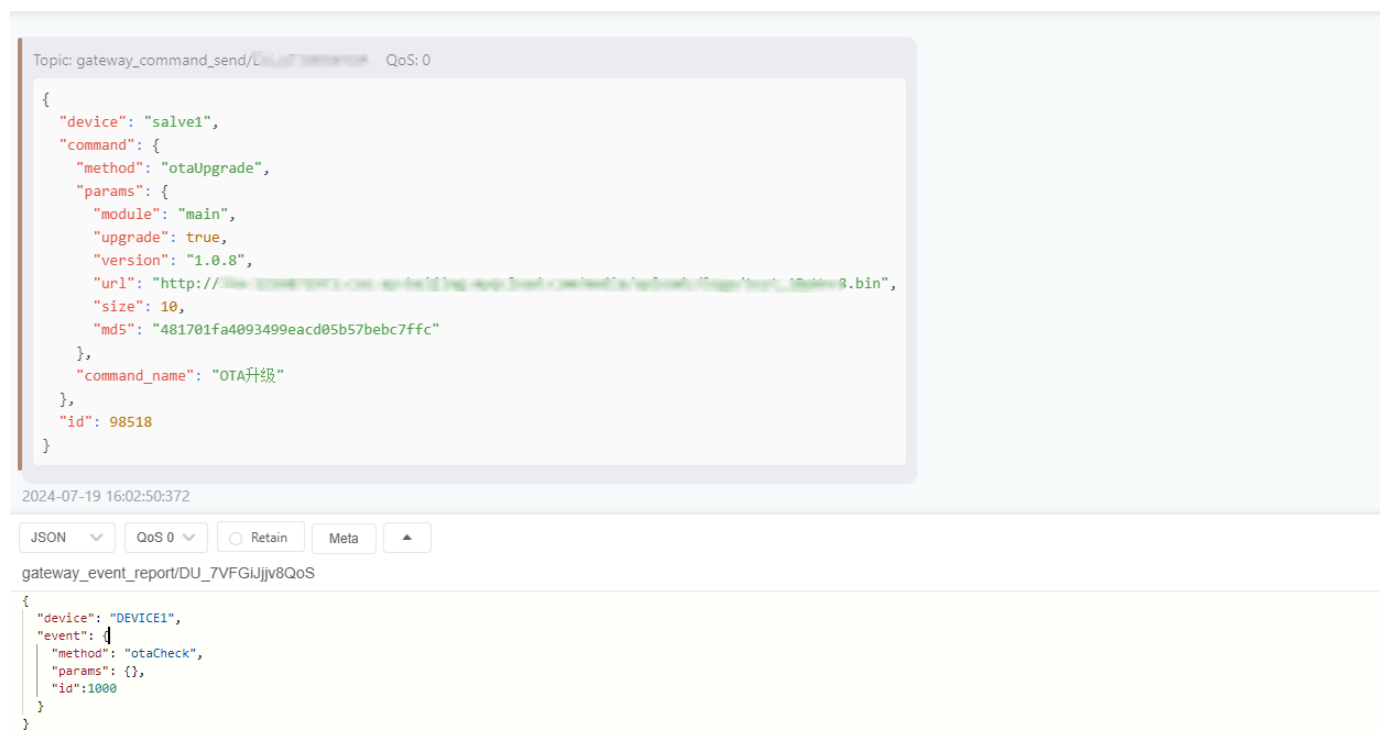
接下来，网关上报子设备的事件，用来检查该子设备是否有新的固件版本可以升级，上报的消息如下：

```
{
  "device": "DEVICE1",
  "event": { "method": "otaCheck", "params": {}, "id": 1000 }
}
```

在 MQTTX 中模拟，如下图：



如果存在新的固件版本，网关会马上收到云平台下发的 OTA 升级推送命令，如下图：



API 概述

API 概述

EZtCloud API 概述

EZtCloud API 为您提供了一个灵活、强大且易于集成的物联网平台，以支持各种行业和企业开发和部署个性化的物联网解决方案，这不仅加速了物联网应用的开发过程，还大大降低了企业开发定制化解决方案的技术门槛和成本。

总的来说，EZtCloud API 为各行业客户打开了通向物联网世界的大门，帮助他们能够以前所未有的方式连接各种硬件设备，推动业务创新和发展。

支持的协议和运行平台

EZtCloud API 几乎可以用于任何应用开发平台，这得益于我们支持多种形式的 API，例如：

- HTTP RESTful API
- HTTP Webhook
- MQTT
- MQTT@Websocket

因此，您可以基于 EZtCloud API 开发出无限扩展的应用软件，包括但不限于以下：

- 基于 Web 浏览器的 SaaS 软件
- 运行在桌面的客户端软件
- 运行在手机端的 App
- 运行在工业平板上的可视化应用
- 运行在展示大屏的应用

HTTPS API

EZtCloud 的 HTTPS API 为定制开发提供了非常全面的资源访问接口，包括对项目、设备类型、设备组、设备信息、设备扩展信息、设备数据、设备通信、设备告警、用户等全面的资源访问，采用 HTTP RESTful API 风格。

第三方应用端可通过 API 主动请求项目内的资产信息和数据，大致具有以下能力：

- 支持服务器端和浏览器端的身份验证。
- 可读取项目信息、设备类型、设备组、设备信息、设备数据、设备扩展信息、设备告警、用户、管理员等信息。
- 可读取设备历史数据。
- 支持设备通信，可向设备下发属性、下发命令、下发自定义数据（支持 JSON / HEX / Plaintext）。

提示

HTTPS API 面向付费用户开放，详细文档请浏览[HTTPS API](#)。

MQTT API

MQTT API 是一种实时消息 API，它允许您的应用软件通过 MQTT 协议来订阅设备的实时消息。应用软件可以是：

- 服务器应用
- Web 前端应用
- App 应用

通过这种实时消息 API，第三方应用可以实时更新设备状态，还可以在服务器应用中接收实时消息，进行更多操作：

- 将设备数据写入自己的数据库
- 将设备数据导出各种自定义的报表形式
- 实现定制化的业务处理
- 实现个性化的设备联动控制
- 实现大数据分析和 AI 模型训练
- 实现更具个性化的 BI 分析和展示

提示

MQTT 应用端订阅功能面向付费用户开放，详细文档请浏览 [MQTT API](#)。

消息规则的流转操作

通过消息规则的流转操作，也可以帮助应用接收实时消息，但相比 MQTT 应用端订阅，这种方式只限于设备属性上报和事件上报的消息流转。

在介绍规则引擎的时候，我们提到了支持数据流转的相关操作，可以用来帮助应用接收实时消息，这些操作包括：

- 转发到 MQTT Broker，详细文档请浏览 [推送到外部 MQTT](#)
- 转发到 URL，详细文档请浏览 [推送到外部URL](#)

HTTPS API

HTTPS API

准备工作

获得项目 API 接入参数

可在 **控制台 > 项目设置 > 应用层API** 中获取 **HTTPS API 接入点**。

全局的，请使用 https 加密访问，为确保数据安全性，全局不支持 http 协议访问。

不同项目的 API 接入点可能不同，请您以控制台获取的地址为准。

此外，您还需要妥善保存此页面的 **ProjectKey** 和 **ProjectToken**（或**ProjectSecret**），用于请求 API 时的身份验证。

认证

应用层应用可通过如下两种方式之一认证：

- 通过**ProjectToken**认证
- 通过**ProjectSecret**认证（不推荐，即将废弃）

	<i>通过ProjectToken认证</i>	<i>通过ProjectSecret认证</i>
是否推荐	推荐	不推荐，即将废弃
安全性	高	低
易用性	一般	高
特点	请求时不直接发送ProjectToken，故 安全性较高	简单易用，但 安全性较低

通过ProjectToken认证

基于timestamp(时间戳)和ProjectToken，通过下文给出的算法计算出signature后，访问其它API时，在请求头中携带前述timestamp、signature和ProjectKey即可。

```
user {ProjectKey}
timestamp {timestamp}
signature {signature}
// 其它 header key
```

signature 的有效期为**2小时**，过期后需重新计算。

ProjectToken应仅保存在应用层服务器，严防泄密。

如下是一段由python语言实现的signature计算代码。其包含详细注释，亦可将其视为签名计算算法的伪代码：

```
import hashlib
import time
# 自 Unix 纪元（January 1 1970 00:00:00 GMT）起的当前时间的秒数。
time_time = time.time()
# 转为字符串，直接忽略小数部分取整。（获取字符串长度为10，例如"1626969655"。）
timestamp = str(time_time)[:10]
# 字符串后拼接token，获取ori_str。假设此处token为"123456"，则ori_str为"1626969655123456"。
ori_str = timestamp + token
# ori_str以utf-8编码，获取encoded_str，接上文例，encoded_str为"1626969655123456"。
encoded_str = ori_str.encode(encoding='utf-8')
# 获取encoded_str的md5码，即为所需签名，接上文例，signature 为"8771d774599c38b15adba116ed82ca8d"。
signature = hashlib.md5(encoded_str).hexdigest()
```

⚠ 注意

实现上述算法后，请务必以上文中的例子验证签名算法的正确性。

通过ProjectSecret认证（不推荐，即将废弃）

通过API获取 AccessToken，用于其它所有 API 请求的身份标识。

AccessToken 获取后的有效期为**24小时**，过期后需重新获取。

在调用其它接口时，将获取到的 token 字符串作为 Bearer Token加入请求头即可。如：

```
Authorization Bearer string_xxx
// 其它 header key
```

服务器发起请求，获取 AccessToken。ProjectSecret应仅保存在应用层服务器，严防泄密。

Request Syntax

```
POST <HTTPS API 接入点>/api_login/ HTTP/1.1
Content-Type: application/json
```

Request Body

```
{
  "username": "project_key",
  "password": "project_secret"
}
```

Body Parameters

- project_key：在控制台项目应用中获取。例如：PR_SrtQt9dEx
- project_secret：在控制台项目应用中获取。

Response Syntax

```
HTTP/1.1 200 OK
Content-type: application/json
```

```
{
  "result": 0,
  "token": "string_xxx"
}
```

HTTP Status Code

- 200: 请求成功。
- 其它: 请求失败。

API 请求代码示例

以获取 API AccessToken 为例，以下是python语言请求 API 的示例代码，仅供参考：

```
import requests
import json

url = '<HTTPS API 接入点>/api/devices/<device_username>/down_attr/'
data = {
  "attrs": "{\"attr1\": \"off\", \"attr2\": 26}"
}
headers = {
  'Content-Type': 'application/json'
}
response = requests.post(url, data=json.dumps(data), headers=headers)

print(response.text)
```

提示

本系统API不限制调用者使用的编程语言，其它编程语言一般也都支持http(s)的调用，这里不再逐一给出样例。

项目资源API

项目下几乎所有资源都可通过API访问。

想快速尝试？我们提供了[在线测试工具](#)。

如下为详细描述：

项目

详见[接口文档](#)。

设备类型

详见[接口文档](#)。

设备组

详见[接口文档](#)。

设备

设备属性下发

Request Syntax

```
POST <HTTPS API 接入点>/api/devices/<device_username>/down_attr/ HTTP/1.1
Content-Type: application/json
```

Path Parameters

- device_username: 设备的username. 例如: DU_dMcDPsiBlk

Request Body

```
{
  "attrs": "{\"attr1\": \"off\", \"attr2\": 26}"
}
```

Body Parameters

- attrs: 一个json字符串。将其load为对象后，其键为设备的属性名称，其值为相应属性的值。支持一次下发多个属性。

Response Syntax

```
HTTP/1.1 200 OK
Content-type: application/json
```

HTTP Status Code

- 200: 请求成功。
- 其它: 请求失败。

其它接口

详见[接口文档](#)。

MQTT API

MQTT API

EZtCloud 支持开放的 MQTT 应用端订阅服务，帮助您在自有软件应用或第三方应用中实时接收设备的最新消息。

您不仅可以在服务器上订阅设备消息，也可以在基于浏览器的 Web 应用中通过 **Javascript** 和 **MQTT@Websocket** 直接订阅设备实时消息，来实现 Web 页面实时更新设备数据，这在开发物联网数据可视化界面时发挥重要的作用。

提示

EZtCloud 提供的 MQTT 应用端订阅服务，与设备的 MQTT 接入并不是同一个 MQTT 服务，请您使用相应的 MQTT 应用端订阅服务地址和主题。

支持订阅哪些消息？

应用端通过 MQTT 应用端订阅可以获得的消息包括：

- 设备属性变化（包括属性上报、属性下发、云端属性更新）
- 设备事件上报
- 设备命令回复
- 设备自定义数据上报
- 设备告警/恢复
- 设备上/下线

支持发布哪些消息？

- 设备属性下发
- 设备命令下发

MQTT 连接认证

在 **项目 > 设置 > MQTT 应用端订阅** 中，可以获得 MQTT 证书，包括：

- projectKey：作为 MQTT username
- projectSecret：作为 MQTT password

MQTT 地址

MQTT 应用端订阅功能对企业版用户开放，请联系技术支持获得详细使用说明。

可在 **控制台 > 项目设置 > 应用层API** 中获取 **MQTT API 接入点**。

全局的，请使用 SSL/TLS 加密访问，为确保数据安全性，全局不支持非加密访问。

不同项目的 API 接入点可能不同，请您以控制台获取的地址为准。

MQTT 订阅主题

以下主题中的字段说明：

- <projectKey>: 项目内唯一，可在 **控制台 > 项目设置 > 应用层API** 中获得。
- <deviceUsername>: 每个设备唯一，可通过设备详情页获得，形如：DU_xxxxxxxxxx。
- +: 表示通配符的主题，可订阅项目中的所有设备。

订阅指定设备属性上报

```
open/<projectKey>/<deviceUsername>/attributes
```

payload

一个json字符串，样例如下：

```
{
  "key1": "{value1}",
  "key2": "{value2}"
}
```

订阅项目内所有设备属性上报

```
open/<projectKey>/+/attributes
```

payload

同 上文 “订阅指定设备属性上报” 。

订阅指定设备事件上报

```
open/<projectKey>/<deviceUsername>/event_report
```

payload

一个json字符串，样例如下：

```
{
  "method": "{name}",
  "params": {
    "key1": "{value1}",
    "key2": "{value2}",
  }
}
```

订阅项目内所有设备事件上报

```
open/<projectKey>/+/event_report
```

payload

同 上文 “订阅指定设备事件上报” 。

订阅指定设备命令回复

```
open/<projectKey>/<deviceUsername>/command_reply
```

payload

一个json字符串，样例如下：

```
{
  "method": "{name}",
  "params": {
    "key1": "{value1}",
    "key2": "{value2}",
  },
  "id": 1
}
```

订阅项目内所有设备命令回复

```
open/<projectKey>/+/command_reply
```

payload

同 上文“订阅指定设备命令回复”。

订阅指定设备自定义数据上报

```
open/<projectKey>/<deviceUsername>/data/<自定义数据流标识符>
```

payload

一个任意字节串，样例如下：

```
fek34j==
```

订阅项目内所有设备自定义数据上报

```
open/<projectKey>/+/data/<自定义数据流标识符>
```

payload

同 上文“订阅指定设备自定义数据上报”。

订阅指定设备告警消息

包含告警触发和告警恢复。

```
open/<projectKey>/<deviceUsername>/alarm
```


payload

一个json字符串，样例如下：

```
{
  // true=告警； false=恢复
  "alerting": true
}
```

订阅项目内所有设备告警消息

包含告警触发和告警恢复。

```
open/<projectKey>/+/alarm
```

payload

同 上文 “订阅指定设备告警消息” 。

订阅指定设备上线/下线通知

```
open/<projectKey>/<deviceUsername>/online
```

payload

一个json字符串，样例如下：

```
{
  // true=上线； false=下线
  "online": true
}
```

订阅项目内所有设备上线/下线通知

```
open/<projectKey>/+/online
```

payload

同 上文 “订阅指定设备上下线通知” 。

MQTT 发布主题

发布指定设备属性下发

```
open/<projectKey>/<deviceUsername>/attributes_push
```

payload

一个json字符串，样例如下：

```
{
  "key1": "{value1}",
  "key2": "{value2}"
}
```

发布指定设备命令下发

```
open/<projectKey>/<deviceUsername>/command_send
```

payload

一个json字符串，样例如下：

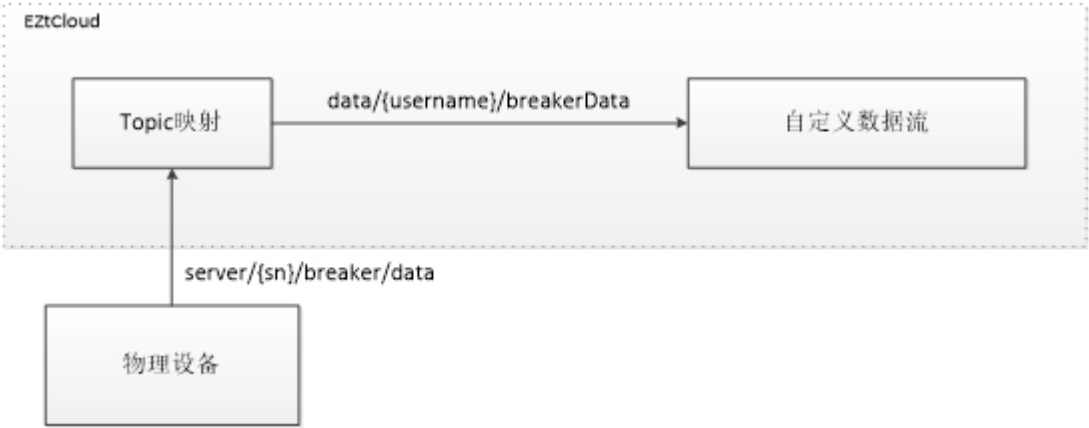
```
{
  "method": "{name}",
  "params": {
    "key1": "{value1}",
    "key2": "{value2}",
  },
  "id": 1
}
```

Topic映射

Topic映射

Topic映射适用于设备无法自定义MQTT topic的场景，需要将设备的topic映射到平台自定义数据流topic上。

举例：某厂商设备不支持配置属性上报topic时，可通过Topic映射功能，将设备上报的数据映射到本系统的自定义数据流的相应topic上。数据流原理图如下所示：



想要实现上图中的功能，只需按如下方式配置一条topic映射即可，如下图所示：

加西亚网关

项目资源

设备管理

设备列表

设备类型

设备分组

规则引擎

定时任务

告警设置

智能场景

Topic映射

维护

项目详情

Topic映射

server/{sn}/breaker/data

查询

所属设备类型	数据流向	自定义数据流	Topic	映射Topic	创建时间	操作
加西亚网关	上行	定时数据上报	data/{username}/breakerData	server/{sn}/breaker/data	2024-10-29 11:24:09	编辑 删除

< 1 >

前往 1 页

⚠ 注意 配置映射topic时, "映射topic"必须包含{sn}, 它是设备的"序列号", 如下图所示:

The image shows a configuration interface for a device. On the left is a sidebar menu with the following items: 设备类型, 设备分组, 规则引擎 (with a function icon), 定时任务 (with a clock icon), 告警设置 (with a bell icon), 智能场景 (with a scene icon), Topic映射 (with a map icon), and 维护 (with a wrench icon). The main area on the right contains the following fields: 启用设备 (with a toggle switch), 设备名称 (with the value '加西亚网关' and an edit icon), 序列号 (with a question mark icon and a red box around it, and an edit icon below it), 设备分组 (with an edit icon), and 设备描述 (with an edit icon).

需要使用映射topic的设备, 都必须配置序列号。